



National University of Computer and Emerging Sciences, Lahore



A.X.I.S. - Architecture & Execution Intelligence Studio

Ahmed Nasir 23L-0530 BS(CS)

Baber Ali 23L-0908 BS(CS)

Abdur Rehman 23L-0694 BS(CS)

Supervisor: Dr. Mirza Mubasher Baig

Final Year Project

October 10, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: A.X.I.S. - Architecture & Execution Intelligence Studio

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: October 10, 2026

Name: Ahmed Nasir

Signature: 

Name: Baber Ali

Signature: 

Name: Abdur Rehman

Signature: 

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

AI coding agents such as Claude Code and Cursor generate code without access to the requirements, decisions and architectural constraints that software teams discuss across Slack, GitHub and project documents, which leads to architectural drift and time-consuming manual review. This report presents A.X.I.S. (Architecture & Execution Intelligence Studio), a system that consolidates this scattered knowledge into a continuously updated traceability graph, keeping only the latest decisions active and linking every rule to its original source. Through a Model Context Protocol server, A.X.I.S. supplies coding agents with relevant project context and validates developer requests against defined constraints, asking the developer for a yes or no confirmation when a violation is found. The report covers the vision, related work, requirements and design of the system.

Executive Summary

Software teams increasingly use AI coding agents such as Claude Code, Cursor and Antigravity to write code. However, requirements, architectural decisions and the planned development order are scattered across Slack, GitHub and project documents. Coding agents cannot see this knowledge, so they often generate code that breaks agreed boundaries. This causes architectural drift and forces technical leads to review the code by hand.

A.X.I.S. (Architecture & Execution Intelligence Studio) acts as a "company brain" for software teams. It collects knowledge from Slack, GitHub, uploaded SRS documents and a visual architecture canvas, and uses a large language model to extract requirements, decisions and constraints. These are stored in a traceability graph, where outdated decisions are marked as superseded and every rule links back to its source. Through a Model Context Protocol (MCP) server, A.X.I.S. gives coding agents the relevant context and checks each developer prompt against the active constraints. If a prompt violates one, the developer answers a yes or no question inside the coding agent, and the decision is recorded.

Existing documentation tools, autonomous agents and traceability platforms need manual input or ignore team decisions. None of them captures informal decisions from team communication and checks coding agent requests against them before code is generated. A.X.I.S. is designed to fill this gap.

The system has four subsystems: a Desktop App built with React, React Flow and Tauri for technical leads and project managers, a RAG Back-End that processes and retrieves project knowledge, an MCP Back-End built with FastMCP that serves coding agents, and a Supabase database using PostgreSQL with pgvector. Context and constraint checks must return within 5 seconds for at least 95% of requests. The system must work with Claude Code, Cursor and Antigravity on Windows, macOS and Linux without a dedicated GPU.

The project is limited by third-party API rate limits, language model costs, the privacy of team conversations, decisions that are never written down, and the lack of real enterprise data for testing. A team of three will deliver the desktop application, a user manual and design documentation over two semesters. It supports Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) by making AI-assisted software development more reliable.

Table of Contents

List of Figures **x**

List of Tables **xi**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.3.1 Definitions 2

1.3.2 Acronyms 2

1.4 Conclusion 3

2 Project Vision **4**

2.1 Problem Domain Overview 4

2.2 Problem Statement 4

2.3 Problem Elaboration 5

2.4 Goals and Objectives 5

2.5 Project Scope 6

2.6 Sustainable Development Goal (SDG) 7

2.7 Constraints 8

2.8 Business Opportunity 8

2.9 Stakeholders Description/User Characteristics 9

2.9.1 Stakeholders Summary 9

2.9.2 Key High-Level Goals and Problems of Stakeholders 9

2.10 Conclusion 10

3 Literature Review / Related Work **11**

3.1 Definitions, Acronyms, and Abbreviations 11

3.1.1 Definitions 11

3.1.2 Acronyms . . . 12
3.2 Detailed Literature Review . . . 12
3.2.1 Eraser.io: Diagrams as Code . . . 12
3.2.2 Miro: The Visual Workspace for Innovation . . . 13
3.2.3 Structurizr: Architecture as Code . . . 14
3.2.4 Tessl: Agent Skills and Context Management . . . 15
3.2.5 Kiro: Spec-Driven Agentic Development . . . 15
3.2.6 AutoGPT: Autonomous Agent Task Sequencing . . . 16
3.2.7 BabyAGI: Task-Driven Autonomous AI . . . 17
3.2.8 Jama Connect: Live Traceability for Requirements . . . 18
3.2.9 Atlassian Rovo: Teamwork Graph for AI Agents . . . 18
3.2.10 Unblocked: Context Layer for AI Coding Agents . . . 19
3.2.11 Graphiti: Temporal Knowledge Graph for Agent Memory . . . 20
3.2.12 Augment Code: Context Engine MCP . . . 21
3.2.13 CodeRabbit: AI Code Review with Team Knowledge . . . 21
3.2.14 Glean: Enterprise Knowledge through MCP . . . 22
3.2.15 ArchUnit: Architecture Rules as Unit Tests . . . 23
3.3 Related Work Summary Table . . . 23
3.4 Conclusion . . . 26

4 Software Requirement Specifications 27
4.1 List of Features . . . 27
4.1.1 Continuous Data Ingestion . . . 27
4.1.2 RAG-Based Traceability Graph . . . 27
4.1.3 MCP Prompt Interception . . . 27
4.1.4 Semantic Architectural Validation . . . 27
4.1.5 Phase-Aware Context Injection . . . 27
4.2 Functional Requirements . . . 27
4.2.1 Account and Access Management . . . 28
4.2.2 Integration and Data Ingestion . . . 28
4.2.3 Architecture and Flow Management . . . 28
4.2.4 MCP Server and AI Prompt Interception . . . 28
4.2.5 Auditing and History . . . 28
4.3 Quality Attributes . . . 29
4.4 Non-Functional Requirements . . . 29

4.4.1 Performance . . . 29
4.4.2 Usability . . . 29
4.4.3 Reliability . . . 29
4.4.4 Security and Privacy . . . 30
4.4.5 Interoperability . . . 30
4.4.6 Scalability . . . 30
4.4.7 Maintainability . . . 30
4.4.8 Portability . . . 30
4.5 Assumptions . . . 30
4.6 Use Cases . . . 30
4.7 Hardware and Software Requirements . . . 40
4.7.1 Hardware Requirements . . . 40
4.7.2 Software Requirements . . . 41
4.8 Graphical User Interfaces . . . 41
4.8.1 Log In . . . 42
4.8.2 Sign Up . . . 42
4.8.3 Dashboard . . . 43
4.8.4 Structural Architecture Canvas . . . 44
4.8.5 Developmental Architecture Canvas . . . 45
4.8.6 Integration Hub . . . 46
4.8.7 Knowledge Hub and Docs . . . 47
4.8.8 Developer Access and MCP . . . 48
4.8.9 Project Settings and Danger Zone . . . 49
4.8.10 Coding Agent IDE Interface . . . 50
4.9 Database Design . . . 51
4.9.1 ER Diagram . . . 51
4.9.2 Data Dictionary . . . 52
4.10 Risk Analysis . . . 59
4.10.1 Technical Risks . . . 59
4.10.2 Programmatic Risks . . . 59
4.10.3 Security Risks . . . 59
4.11 Conclusion . . . 59

5 High-Level and Low-Level Design 61

5.1 System Overview . . . 61

5.1.1 Continuous Data Ingestion Engine . . . 61
5.1.2 Traceability Graph and Vector Storage . . . 61
5.1.3 Model Context Protocol (MCP) Server . . . 61
5.1.4 Visual Architecture Studio . . . 61
5.2 Design Considerations . . . 62
5.2.1 Assumptions and Dependencies . . . 62
5.2.2 General Constraints . . . 62
5.2.3 Goals and Guidelines . . . 62
5.2.4 Development Methods . . . 63
5.3 System Architecture . . . 63
5.3.1 Subsystem Architecture . . . 64
5.4 Architectural Strategies . . . 66
5.4.1 Choice of Technologies . . . 66
5.4.2 Reuse of Existing Software Components . . . 66
5.4.3 External Database and Data Storage Management . . . 66
5.4.4 Communication Mechanisms . . . 67
5.4.5 Concurrency and Synchronization . . . 67
5.4.6 User Interface Paradigms . . . 67
5.4.7 Error Detection and Recovery . . . 67
5.4.8 Future Plans for Extension . . . 67
5.4.9 Version Control . . . 68
5.5 Class Diagram . . . 68
5.6 Sequence Diagrams . . . 69
5.6.1 Standard User Authentication . . . 69
5.6.2 Document Parsing and Knowledge Extraction . . . 70
5.6.3 Continuous Data Ingestion . . . 70
5.6.4 Visual Architecture to Constraint Generation . . . 72
5.6.5 MCP Client Authentication and Handshake . . . 73
5.6.6 AI Prompt Interception and Semantic Validation . . . 74
5.7 Policies and Tactics . . . 75
5.7.1 Hybrid Retrieval with Graph Expansion . . . 75
5.7.2 Coding Guidelines and Conventions . . . 75
5.7.3 MCP Tool Interface Design . . . 75
5.7.4 Prompt and Model Management . . . 75

5.7.5 Testing Strategy 75
5.7.6 Security and Secrets Management 76
5.7.7 Source Code Organization 76
5.8 Conclusion 76

List of Figures

2.1 SDG 9: Industry, Innovation and Infrastructure 7

4.1 Log In Screen 42
4.2 Sign Up Screen 42
4.3 Dashboard Screen 43
4.4 Structural Architecture Canvas 44
4.5 Developmental Architecture Canvas 45
4.6 Integration Hub Screen 46
4.7 Knowledge Hub and Docs Screen 47
4.8 Developer Access and MCP Screen 48
4.9 Project Settings Screen 49
4.10 Coding Agent IDE with A.X.I.S. MCP Interception 50
4.11 ER Diagram 51

5.1 Architecture Diagram of A.X.I.S. 63
5.2 Class Diagram 68
5.3 Standard User Authentication 69
5.4 Document Parsing and Knowledge Extraction 70
5.5 Continuous Data Ingestion 71
5.6 Visual Architecture to Constraint Generation 72
5.7 MCP Client Authentication and Handshake 73
5.8 AI Prompt Interception and Semantic Validation 74

List of Tables

2.1 Problem Statement describing the problem, effect, and proposed solution. 4

3.1 Related Work Summary: Architecture, Documentation and Traceability Tools . . . 24
3.2 Related Work Summary: AI Agents and Context Tools 25

4.1 Use case: Sign Up 31
4.2 Use case: Log In 32
4.3 Use case: Create New Project 32
4.4 Use case: Manage Project Settings 33
4.5 Use case: Manage Developer Access 34
4.6 Use case: Design Structural Architecture 35
4.7 Use case: Design Developmental Architecture 36
4.8 Use case: Upload SRS Documents 37
4.9 Use case: Connect Slack Workspace 38
4.10 Use case: Connect GitHub Repository 39
4.11 Use case: Connect MCP with Coding Agent 40
4.12 Organization Table 52
4.13 User Table 52
4.14 Project Table 52
4.15 Project Member Table 53
4.16 API Key Table 53
4.17 Integration Table 54
4.18 Source Table 54
4.19 Evidence Table 55
4.20 Knowledge Node Table 56
4.21 Trace Link Table 57
4.22 Evidence Link Table 57
4.23 Drift Alert Table 57
4.24 HITL Decision Table 58

Chapter 1 Introduction

In a world where software engineering depends more and more on self-directed coding agents and AI-assisted rapid development, it is critical to keep integrating business rules and architectural restrictions. The purpose of this chapter is to provide a quick overview of how the dependability of AI-generated code may help achieve development goals that are in line with corporate objectives. The need to consider organizational knowledge as a core input to the architecture is becoming clear as distributed team communication use cases, like Slack or GitHub, expand alongside isolated agent use. MCP server ("company brain") and semantic architecture linter are proposed in order to resolve the issues brought about by disconnected agentic prompts, while subsequently creating possibilities for transforming an abundance of disconnected data into a traceability graph and enabling developers to utilize Human-in-the-Loop (HITL) verification.

1.1 Purpose of this Document

The purpose of this document is to present the requirements, design, and development of "A.X.I.S.". In software teams, requirements, decisions, and constraints are scattered across Slack conversations, GitHub pull requests, and individual team members, and AI coding agents such as Claude Code and Cursor have no access to them, so they often generate code that contradicts what the team has already decided. A.X.I.S. aims to solve this by consolidating this knowledge into a traceability graph, keeping it up to date as discussions evolve, and providing it to coding agents through an MCP server. When a developer's request conflicts with an established constraint, the system alerts them through a human-in-the-loop confirmation before proceeding. The intended outcome is a functional minimum viable product, and this document describes how it is designed and built.

1.2 Intended Audience

The intended audience of this report includes:

- FYP Evaluation Panel and Project Supervisor evaluating the project's requirements, architectural design, and implementation.
- Software Developers using AI coding tools (e.g., Claude Code, Cursor, GitHub Copilot) who require them to strictly follow team rules and decisions.
- Software Architects and Technical Leads seeking to understand how A.X.I.S. captures architectural decisions from Slack and GitHub to maintain consistency in generated code.

- Future Developers and Researchers referencing this document to extend A.X.I.S. with new knowledge sources, integrations, or features.

1.3 Definitions, Acronyms, and Abbreviations

The definitions and acronyms used in this report are listed below.

1.3.1 Definitions

Company Brain: A conceptual framework for the MCP server that works as a centralized library of organizational knowledge to govern and align AI-generated code with company objectives.
Semantic Architecture Linter: A validation engine that checks coding agent requests against the team's recorded decisions and architectural constraints, and alerts the developer when a request violates them.
Traceability Graph: A systematic mapping that transforms fragmented and disjointed organizational communication data (e.g., from Slack or GitHub) into a coherent relationship model for development verification.
Structural Awareness: The capacity of an AI agent or system to comprehend and respect the architectural boundaries, component relationships, and established design constraints within a software project.
Temporal Awareness: The ability of an AI agent to understand and follow the planned order of development phases and tasks.

1.3.2 Acronyms

AI: Artificial Intelligence
API: Application Programming Interface
A.X.I.S.: Architecture & Execution Intelligence Studio
FYP: Final Year Project
HITL: Human-in-the-Loop
LLM: Large Language Model
MCP: Model Context Protocol
RAG: Retrieval-Augmented Generation
SDG: Sustainable Development Goal
SRS: Software Requirements Specification

1.4 Conclusion

A.X.I.S. is presented in the first chapter as a remedy for AI coding agents producing code that conflicts with team decisions dispersed among GitHub, Slack, and team members. The project's vision is presented in the second chapter, which also explains how A.X.I.S. compiles this information into a traceability graph for software houses, project managers, and technical leads. The third chapter highlights the gaps that A.X.I.S. fills by reviewing previous research on AI coding agents, retrieval-augmented generation, and project knowledge management. The requirements of A.X.I.S. are outlined in the fourth chapter, which covers everything from creating the architecture canvas and integrating Slack and GitHub to using Human-in-the-Loop confirmation to validate developer prompts. The design of A.X.I.S. is explained in the fifth chapter, which also demonstrates how the Desktop App, RAG Back-End, MCP Back-End, and Supabase database collaborate to provide coding agents with verified context.

Chapter 2 Project Vision

The problem statement, project scope, and goals are explained in the section that follows, along with the business opportunity and stakeholder list.

2.1 Problem Domain Overview

This project automates software architecture management as well as task sequencing throughout development. By validating developer requests before code is generated, it helps AI coding agents adhere to project phases and structural rules. The system extracts knowledge from organizational data into a structured traceability graph and uses Retrieval-Augmented Generation (RAG) to retrieve the relevant parts of it. This graph outlines the precise sequence of development tasks and maps system components. The solution directly provides AI agents with accurate, phase-specific context by combining an automatic validator with a Model Context Protocol (MCP) server. This reduces manual rule checking while keeping the developer in control, since any request that conflicts with a team decision needs the developer's confirmation.

2.2 Problem Statement

A detailed problem statement is shown in Table 2.1.

Table 2.1: Problem Statement describing the problem, effect, and proposed solution.

Problem	Autonomous AI coding agents function independently of project execution stages, organizational knowledge, and structural rules. They are unable to understand the necessary order of development activities or system limits since they do not have access to ongoing team conversations and architectural decisions.
Effect	AI agents produce code that introduces architectural drift, disregards necessary development sequences, and violates system constraints. This results in technical debt and requires engineers to manually check AI-generated code against project specifications, which takes a lot of time.
Solution	A.X.I.S. builds a traceability graph of the latest requirements, decisions and constraints from Slack, GitHub, documents and an architecture canvas. Through an MCP server, it gives coding agents validated context before code is generated and asks the developer for a yes or no confirmation when a request violates a constraint.

2.3 Problem Elaboration

This project's primary problem is the architectural drift and execution misalignment caused by AI coding agents working without the team's context. Even while contemporary agentic tools are excellent at producing syntactically accurate code, they function independently of an organization's dynamic architectural choices, business regulations, and project execution stages.

Through collaborative tools like Slack and GitHub, development teams constantly negotiate structural limits (like database access restrictions, API gateway usage) and work sequencing (like creating data schemas before frontend interfaces). AI agents often produce code that breaks internal limitations or tries to construct components out of the necessary developmental order since they do not have real-time access to this ongoing agreement. Software engineers and technical leads have the heavy duty of manually reviewing and refining AI-generated code to make sure it complies with organizational rules as a result of this disconnect, which also creates substantial technical debt and breaks the system's rules.

To mitigate these challenges, A.X.I.S. proposes a continuous architectural governance layer that transforms scattered organizational knowledge into a structured traceability graph. By intercepting prompts via a Model Context Protocol (MCP) server, the system ensures AI generation is both structurally and temporally validated.

To fully address this domain, the project will tackle the following sub-problems:

- Project knowledge is scattered across Slack conversations, GitHub and documents.
- Informal team discussions are hard to turn into structured, searchable rules.
- Coding agents do not know component boundaries or the planned development order.
- Generated code is checked against team decisions only through slow manual review.

2.4 Goals and Objectives

Our project goals are as follows:

- Develop a fully functional MCP server that acts as a "company brain" for software teams.
- Build a visual canvas to design and define the system architecture.
- Convert this scattered information into a structured traceability graph.
- Ensure only the latest decisions are used, marking outdated ones as superseded.
- Supply relevant context automatically to coding agents like Claude Code, Cursor, and Antigravity.

- Detect when a developer's request or generated code violates a defined constraint.
- Alert the developer through Human-in-the-Loop confirmation before a violating change is made.
- Improve the accuracy and reliability of AI-generated code in real software projects.

Our objectives behind this project are to:

- Design a semantic layer to extract requirements, decisions, and constraints from communication.
- Build a retrieval system that fetches the most relevant context for a given developer prompt.
- Link requirements, decisions, and architecture together in one graph.
- Maintain traceability so every rule can be traced back to its original source.
- Develop a semantic linter to validate prompts and code changes against team constraints.
- Keep a record of every violation and override decision made by developers.
- Make the system work across multiple coding agents instead of a single one.
- Build a scalable backend infrastructure for storing and retrieving organizational knowledge.

2.5 Project Scope

We will develop A.X.I.S., a system that collects scattered project knowledge from Slack, GitHub, uploaded SRS documents and visual architecture canvases, organizes it into a traceability graph, and delivers it to AI coding agents through an MCP server so that generated code follows the team's actual requirements and decisions. The system consists of three main parts: a React Flow canvas where architects define the intended system architecture, a backend that ingests team communication and converts it into a structured and continuously updated knowledge graph, and an MCP server with a semantic architecture linter that supplies context to coding agents and validates their work against defined constraints. The system will allow users to:

- Design the system architecture visually on a canvas.
- Connect their Slack workspace and GitHub repository as knowledge sources.
- Automatically extract requirements, decisions and constraints from team discussions.
- Upload SRS and other requirement documents as knowledge sources.
- Keep only the latest decisions active and mark outdated ones as superseded.
- Trace any requirement or constraint back to its original source.

- Receive relevant project context automatically inside their coding agent.
- Get a Human-in-the-Loop confirmation when a request violates a constraint.
- Keep a record of all violations and developer override decisions for traceability.

The backend and MCP server will be implemented in Python using FastAPI and the MCP Python SDK, with PostgreSQL and pgvector for storing the traceability graph and embeddings. The architecture canvas will be built in React using React Flow. Integration with coding agents will be done through MCP and the agents' hook interfaces.
At the end of the project we will deliver:

- The A.X.I.S. web app wrapped as a standalone desktop application using Tauri.
- A user manual for the system.
- Documentation of the design, implementation and testing process.

The project scope does not include the following:

- Providing hosting or server infrastructure for the system.
- Supporting knowledge sources other than Slack, GitHub, uploaded documents, and the canvases.
- Supporting coding agents that do not provide MCP or hook interfaces.

2.6 Sustainable Development Goal (SDG)

Our sustainable development goal is SDG 9: Industry, Innovation and Infrastructure, shown in Figure 2.1. A.X.I.S. supports this goal by strengthening software development infrastructure, making AI-assisted development more reliable and consistent for software organizations.



Figure 2.1: SDG 9: Industry, Innovation and Infrastructure

2.7 Constraints

The following are the constraints to the system:

- **Third-Party API Limits:** Slack and GitHub restrict how often their APIs can be called and which data can be accessed. Access also requires explicit permission from the workspace or repository owner.
- **Differences Between Coding Agents:** Claude Code, Cursor, and Google Antigravity each implement MCP and handle user prompts differently, requiring the prompt interception and semantic validation mechanisms to be designed to work consistently across all platforms.
- **LLM Cost and Context Limits:** Extracting knowledge and checking constraints depend on large language model APIs, which incur usage costs and enforce strict token limits on the amount of context that can be sent in a single request.
- **Data Privacy:** Team conversations may contain confidential business information, requiring the system to store and process this data securely and exclusively with the organization's consent.
- **Incomplete Knowledge:** Architectural decisions and execution plans made verbally in meetings or calls, and never written in Slack or GitHub, cannot be captured or enforced by the system.
- **Testing Data Availability:** Real enterprise Slack and GitHub data is difficult to obtain for testing purposes, so the system will be developed and demonstrated partially using prepared sample data.
- **Time and Resource Limitations:** The project must be completed within two academic semesters by a team of three members.

2.8 Business Opportunity

Software development is being accelerated by the quick uptake of autonomous AI coding agents like Claude Code and Cursor, but this is also creating a crucial operational bottleneck: making sure AI-generated output complies with established business standards and architectural choices. Costly rework, project delays, and compounding technical debt result when these agents function independently, disregarding structural boundaries and temporal project phases. A continuous architectural governance platform that easily links a team's organizational knowledge to its AI operations is thus presented with a compelling business potential. A.X.I.S. reduces the need for manual rule verification by directly extracting requirements from collaborative platforms like Slack and GitHub and feeding this validated context into AI agents via a Model Context Protocol (MCP) server. Software companies may improve the effectiveness and return on investment of their AI-assisted development pipelines thanks to this automated

alignment, which also reduces architectural drift and technical debt.

2.9 Stakeholders Description/User Characteristics

The primary stakeholders and users of A.X.I.S. are as follows:

- Software Developers
- Software Architects and Technical Leads
- Project Managers
- Software Houses

2.9.1 Stakeholders Summary

Software developers are the main users of the system. They use AI coding agents such as Claude Code, Cursor and Antigravity in their daily work. A.X.I.S. supplies their agent with relevant project context and alerts them through a Human-in-the-Loop confirmation when a request violates a constraint, leaving the final decision with them.

Software architects and technical leads define the system architecture on the A.X.I.S. canvas and make key decisions through Slack and GitHub discussions. They rely on A.X.I.S. to ensure these decisions are followed in the code produced by developers and their agents.

Software houses are the organizations that adopt A.X.I.S. for their projects. They need their teams to deliver code that meets project requirements, and benefit from more accurate and consistent development.

Project managers set up projects, connect Slack and GitHub, and generate API keys for developers. They use A.X.I.S. to keep development aligned with the planned order of work.

2.9.2 Key High-Level Goals and Problems of Stakeholders

2.9.2.1 Centralized Project Knowledge

Project requirements and decisions are scattered across Slack and GitHub. A.X.I.S. brings them together in a single traceability graph, removing the need to search multiple sources.

2.9.2.2 Consistent AI-Generated Code

AI coding agents have no access to team decisions and often generate code that contradicts them. A.X.I.S. provides them the relevant context so generated code follows the defined architecture and requirements.

2.9.2.3 Traceability of Decisions

Teams often cannot tell why a rule exists or whether it is still valid. A.X.I.S. links every rule to its original source and keeps only the latest decisions active.

2.9.2.4 Developer Control through Human-in-the-Loop

Strict enforcement can block legitimate work, while no enforcement lets violations go unnoticed. A.X.I.S. alerts developers to violations, lets them decide whether to proceed, and records every override.

2.10 Conclusion

The fundamental vision and problem domain of A.X.I.S. were described in this chapter. It outlined the goals, objectives, and limitations that guide the system's development, as well as the problem statement and problem elaboration, emphasizing the difficulties of architectural drift brought on by isolated AI coding agents. A.X.I.S. was described in the project scope as a continuous governance layer that includes a Model Context Protocol (MCP) server driven by Retrieval-Augmented Generation (RAG), a visual architectural canvas, and a knowledge extraction backend. The system's capacity to remove human verification bottlenecks, lower technical debt, and preserve a centralized traceability graph from dispersed organizational data was highlighted in the business opportunity and stakeholder descriptions, which bolstered the major project justification. The chapter concluded by outlining the main users who will gain from human-in-the-loop architectural enforcement and identifying the project's alignment with the Industry, Innovation, and Infrastructure Sustainable Development Goal.

Chapter 3 Literature Review / Related Work

This chapter offers a thorough analysis of current platforms and technologies for visual collaboration, architectural documentation, and AI-driven software development. To determine present capabilities and emphasize the functional gaps that A.X.I.S. seeks to fill, the literature is examined. This research demonstrates the need for ongoing architectural governance, execution routines, and dynamic prompt interception through the Model Context Protocol (MCP) by looking at both static architectural tools and modern agentic settings.

3.1 Definitions, Acronyms, and Abbreviations

The definitions and acronyms used in this chapter, in addition to those listed in Chapter 1, are given below.

3.1.1 Definitions

Agentic Development Environment: A development tool in which an AI agent can plan, write, test and modify code across multiple files with minimal human intervention, such as Kiro.

Agent Skill: A versioned package of instructions, documentation and rules that tells an AI agent how to perform a task or behave in a specific codebase or organization.

Architecture Drift: The gradual divergence of a system's implemented code from its intended architecture and agreed design decisions.

C4 Model: A model for describing software architecture at four levels of detail: system context, containers, components and code.

Human-in-the-Loop: A design approach in which a human must review or approve a decision made by an AI system before it takes effect.

Live Traceability: Traceability that is updated continuously as artifacts change, rather than being compiled manually at the end of a project.

Model Context Protocol: An open standard that allows AI applications to connect to external tools and data sources through a common interface.

Property-Based Testing: A testing technique that checks whether code satisfies general properties across many automatically generated inputs, rather than a small number of fixed examples.

Retrieval-Augmented Generation: A technique in which relevant information is first retrieved from a knowledge source and then provided to a large language model, so that its response is based on that information.

Spec-Driven Development: An approach in which a structured specification of requirements and de-

sign is written before any code, and AI agents then generate code from that specification.

Steering Files: Files in Kiro that store persistent project knowledge, such as coding conventions and technology choices, which the agent reads in every session.

Task Decomposition: The process of breaking a high-level goal into smaller sub-tasks that can be executed one at a time.

Teamwork Graph: Atlassian's data layer that connects people, work items, code and content across its products and connected applications.

Trace Score: A measure in Jama Connect of how complete the traceability links in a project are.

Traceability: The ability to follow the links between a requirement and its related artifacts, such as design decisions, code and tests.

3.1.2 Acronyms

AGI: Artificial General Intelligence (as used in the name BabyAGI)

AWS: Amazon Web Services

C4: Context, Containers, Components and Code

CLI: Command-Line Interface

DSL: Domain-Specific Language

GPT: Generative Pre-trained Transformer (as used in the name AutoGPT)

IDE: Integrated Development Environment

NLP: Natural Language Processing

PR: Pull Request

3.2 Detailed Literature Review

This section includes a detailed literature review of the problem area, categorizing prior work and existing development tools relevant to structural architecture management and AI coding assistants.

3.2.1 Eraser.io: Diagrams as Code

3.2.1.1 Summary of the research item

Eraser.io [1] is a documentation and diagramming tool engineered specifically for software development teams. It utilizes a "Diagrams as Code" methodology, allowing developers to generate architecture diagrams, cloud topologies, and sequence flows directly from markdown-like syntax. The platform

integrates seamlessly with GitHub to keep documentation version-controlled alongside the codebase, providing a centralized hub for engineering teams to visually map system components and API boundaries.

3.2.1.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Allows teams to rapidly generate system topologies.
- Stores diagrams as text, allowing them to be version-controlled alongside the code.
- Produces static diagrams that are not checked against the actual code, making them outdated.
- Lacks automated enforcement of structural boundaries on AI agents.

3.2.1.3 Relationship to the proposed research work

This tool provides a foundational understanding of how structural boundaries can be defined programmatically. While Eraser.io focuses on passive documentation, A.X.I.S. extends this concept by transforming structural definitions into active governance. Instead of merely drawing the architecture, A.X.I.S. stores it in a traceability graph and checks agent requests against it, ensuring AI agents respect the structural boundaries that Eraser.io would typically only visualize.

3.2.2 Miro: The Visual Workspace for Innovation

3.2.2.1 Summary of the research item

Miro [2] is a widely adopted digital whiteboard platform designed to facilitate freeform collaborative innovation and team brainstorming. In the context of software engineering, it is frequently used by architects, product managers, and developers to visually negotiate system requirements, map out user journeys, and establish initial architectural consensus before formal development begins. The platform supports real-time multi-user editing and, since 2026, provides an MCP server that allows AI coding tools to read board content [3].

3.2.2.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Exceptional flexibility for unstructured, cross-functional collaboration.
- Highly effective for capturing initial, high-level organizational intent.

- Board content is mostly freeform and visual, so decisions are not stored as a structured model.
- Agents can read board content, but they don't check requests against the agreed decisions.

3.2.2.3 Relationship to the proposed research work

Miro exemplifies how teams naturally communicate and establish consensus. A.X.I.S. addresses the exact limitation of platforms like Miro by capturing similar unstructured team discussions directly from collaborative tools (e.g., Slack and GitHub). A.X.I.S. uses Retrieval-Augmented Generation (RAG) to store this consensus in a searchable form, so that relevant decisions can be retrieved and checked when AI agents generate code.

3.2.3 Structurizr: Architecture as Code

3.2.3.1 Summary of the research item

Structurizr [4] is a tool that implements the C4 model for visualizing software architecture. It uses an "Architecture as Code" Domain-Specific Language (DSL) to describe a software system at the context, container and component levels. Since the architecture is defined as a single text-based model, Structurizr can generate multiple consistent views from one version-controlled source of truth.

3.2.3.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this tool that we found:

- Follows a formal structural model, which establishes clear boundaries between components.
- Avoids the ambiguity and inconsistency common in hand-drawn architecture diagrams.
- Needs manual DSL updates by developers, so it easily gets outdated.
- Does not use informal team discussions as a source of decisions and lacks temporal awareness.

3.2.3.3 Relationship to the proposed research work

Structurizr shows the value of keeping architecture in a single source of truth rather than in disconnected diagrams. A.X.I.S. follows the same principle, but builds and updates its architectural knowledge automatically from decisions made in Slack and GitHub instead of manual DSL edits. Through its semantic architecture linter, A.X.I.S. then checks user requests against these component boundaries, so that the architecture is actively applied during development rather than only documented.

3.2.4 Tessl: Agent Skills and Context Management

3.2.4.1 Summary of the research item

Tessl [5] began as a platform for spec-driven AI development and has since evolved into an agent enablement platform. It manages agent skills, which are versioned packages of instructions, documentation and rules that tell AI agents how to behave in a specific codebase or organization. Teams can install skills from a public registry through a CLI, publish their own, and evaluate whether each skill improves agent performance. The platform supports MCP and works across different agents and models.

3.2.4.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Provides reusable, versioned context that works across different AI agents and models.
- Evaluates skills, allowing teams to measure whether a skill actually improves agent behaviour.
- Skills are written and updated manually, so they only cover explicitly documented knowledge.
- Skills act as static instructions rather than a live, linked model of the project's current state.

3.2.4.3 Relationship to the proposed research work

Tessl confirms the growing need for organization-specific context that works across multiple AI agents, and A.X.I.S. shares its vendor-neutral, MCP-based approach. However, instead of relying on hand-written skills, A.X.I.S. builds its knowledge automatically from decisions made in Slack and GitHub and links it in a traceability graph that stays aligned with the project's live status. A.X.I.S. also checks agent requests against this knowledge with developer confirmation, rather than only providing instructions.

3.2.5 Kiro: Spec-Driven Agentic Development

3.2.5.1 Summary of the research item

Kiro [6] is an agentic development environment by Amazon Web Services (AWS), available as an Integrated Development Environment (IDE) and a Command-Line Interface (CLI). It follows a spec-driven approach in which the developer's prompt is first converted into requirements, design and task documents, which Kiro then implements autonomously. Project conventions can be stored in steering files, and the generated code is verified against the specification using property-based tests.

3.2.5.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Highly autonomous execution greatly accelerates feature development.
- Structured specifications make multi-step tasks reviewable before code is written.
- Specifications are derived top-down from a single developer's prompt rather than team consensus.
- Specifications are static files that do not stay in sync with team discussions or live project status.

3.2.5.3 Relationship to the proposed research work

Kiro demonstrates the value of structured specifications in guiding autonomous agents. A.X.I.S. differs by extracting requirements and decisions bottom-up from team channels such as Slack and GitHub, keeping them synchronized organization-wide through issue and Pull Request (PR) status, and checking agent requests against team-wide rules and cross-developer dependencies rather than a single prompt-derived specification. Since Kiro supports the Model Context Protocol (MCP), A.X.I.S. can supply this context to it directly.

3.2.6 AutoGPT: Autonomous Agent Task Sequencing

3.2.6.1 Summary of the research item

AutoGPT [7] is an open-source autonomous agent framework that enables Large Language Models (LLMs) to execute multi-step objectives independently. Instead of relying on continuous, step-by-step human prompting, users provide AutoGPT with a high-level goal. The framework then attempts to autonomously decompose this goal into a sequence of smaller sub-tasks, execute them, and evaluate the outcomes to determine the next step.

3.2.6.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this framework that we found:

- Demonstrates the foundational ability of AI to decompose large objectives into actionable steps.
- Eliminates the need for continuous human-in-the-loop micro-management during task execution.
- Can repeat steps or follow an incorrect sequence, as it relies only on the LLM's own planning.
- Lacks access to roadmaps and business rules unless they're provided manually.

3.2.6.3 Relationship to the proposed research work

AutoGPT shows that autonomous agents attempt to plan the order of their own tasks, but without knowledge of the organization's roadmap this order may not match the team's plans. A.X.I.S. addresses this limitation. Instead of leaving the agent to guess the execution order, A.X.I.S. extracts the planned order of work from collaborative platforms such as Slack and GitHub and supplies it to the agent through the Model Context Protocol (MCP). This helps keep autonomous execution aligned with the team's intended development flow.

3.2.7 BabyAGI: Task-Driven Autonomous AI

3.2.7.1 Summary of the research item

BabyAGI [8] is a task-driven AI orchestration script that focuses heavily on dynamic task prioritization. Its core loop consists of pulling the highest-priority task from a generated list, executing it via an LLM, evaluating the result, and subsequently generating and reprioritizing new tasks based on that outcome. This creates a continuous cycle of planning and execution driven entirely by the LLM's own reasoning.

3.2.7.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this framework that we found:

- Implements a simple, continuous loop for dynamic task generation and prioritization.
- Breaks down complex objectives without requiring predefined workflows.
- Reprioritization relies only on LLM logic, ignoring team consensus and issue trackers.
- Lacks structural awareness, so it might prioritize tasks that break architectural rules.

3.2.7.3 Relationship to the proposed research work

BabyAGI shows the risk of letting an AI agent decide the order of its own work, as its priorities come from general LLM reasoning rather than organizational intent. A.X.I.S. addresses this by providing agents with a traceability graph built from real team discussions in Slack and GitHub, including the dependencies between requirements. In this way, the order of development is guided by the team's validated decisions rather than the agent's own assumptions.

3.2.8 Jama Connect: Live Traceability for Requirements

3.2.8.1 Summary of the research item

Jama Connect [9] is a requirements, risk and test management platform used by organizations developing complex and often regulated products. Its Live Traceability feature keeps requirements, tests, defects and risks linked in real time, and measures how complete these links are through a Trace Score. Its AI assistant, Jama Connect Advisor, uses Natural Language Processing (NLP) to check requirement quality and recommend missing relationships, and the platform now provides a Model Context Protocol (MCP) server for AI tools [10].

3.2.8.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Continuous traceability measurement exposes missing links early rather than during audits.
- AI suggestions are reviewed by humans, keeping them in control of every traceability decision.
- Requires requirements to be formally entered into the tool, demanding significant manual effort.
- Does not capture informal decisions from team communication or detect architecture drift in code.

3.2.8.3 Relationship to the proposed research work

Jama Connect demonstrates the value of a live traceability model in which every artifact is linked to its source. A.X.I.S. adopts this idea but applies it to software teams whose decisions are made informally in Slack and GitHub and are rarely recorded in formal requirements tools. Instead of linking compliance evidence, A.X.I.S. turns these decisions into architectural constraints that coding agents consult before generating code.

3.2.9 Atlassian Rovo: Teamwork Graph for AI Agents

3.2.9.1 Summary of the research item

Rovo [11] is Atlassian's AI layer across Jira and Confluence, combining enterprise search, conversational AI and autonomous agents. It is powered by the Teamwork Graph, which connects people, work items, code and content across Atlassian and connected applications. Through the Rovo MCP Server and the Teamwork Graph Command-Line Interface (CLI), both in open beta, this context is now available to external coding agents such as Claude Code [12].

3.2.9.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Provides organization-wide context to AI agents through an open standard.
- Builds on a very large existing user base and integration ecosystem.
- Models general work items and documents rather than architectural decisions and constraints.
- Doesn't validate against architectural rules or detect drift, and requires the Atlassian ecosystem.

3.2.9.3 Relationship to the proposed research work

Rovo is one of the closest existing system to A.X.I.S., confirming that delivering organizational context to coding agents through MCP is a growing industry direction. A.X.I.S. differs by modeling architecture-specific knowledge such as decisions, constraints and components, checking agent requests against them with developer confirmation, and detecting drift. It is also independent of any single vendor's ecosystem, making it suitable for teams that do not use Atlassian products.

3.2.10 Unblocked: Context Layer for AI Coding Agents

3.2.10.1 Summary of the research item

Unblocked [13] describes itself as the context layer for agentic software development. It connects to around thirty engineering tools, including GitHub, Slack, Jira, Confluence and Notion, and makes their combined knowledge available to developers and AI agents. Through an MCP server, coding agents such as Claude Code, Cursor and GitHub Copilot can search this knowledge and receive answers with citations to their sources. Unblocked also offers AI code review that considers team standards and past decisions, along with pull request summaries and failure triage.

3.2.10.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Brings knowledge from many team tools into one place for both developers and AI agents.
- Cites the source of every answer, making the context easy to verify.
- Read-only MCP forces manual context search instead of upfront validation.
- Does not model the intended system architecture or the planned order of development.

3.2.10.3 Relationship to the proposed research work

Unblocked confirms the central idea behind A.X.I.S.: that coding agents need context from team communication such as Slack and GitHub, delivered through MCP. A.X.I.S. goes a step further by turning this knowledge into explicit constraints, linking them to a visual structural and developmental architecture, and checking developer requests against them with a yes or no confirmation before code is generated, rather than only reviewing code after it is written.

3.2.11 Graphiti: Temporal Knowledge Graph for Agent Memory

3.2.11.1 Summary of the research item

Graphiti is the open-source knowledge graph engine at the core of Zep, a memory layer for AI agents [14]. It builds a knowledge graph incrementally from incoming data and records when each fact became valid and when it stopped being valid. When a fact changes, the older fact is marked as invalid rather than overwritten, so the history remains available. Graphiti retrieves knowledge using hybrid search combined with graph traversal, without calling a language model at query time. In its evaluation, Zep outperformed MemGPT on the Deep Memory Retrieval benchmark (94.8% against 93.4%).

3.2.11.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this framework that we found:

- Keeps knowledge current by invalidating outdated facts while preserving their history.
- Updates the graph incrementally and retrieves results quickly using hybrid search.
- General-purpose memory lacking awareness of requirements, architecture, or code.
- Requires a language model call for every write and a separate graph database such as Neo4j.

3.2.11.3 Relationship to the proposed research work

Graphiti's approach of invalidating outdated facts instead of deleting them supports the design of A.X.I.S., in which superseded decisions remain in the traceability graph but are no longer used for checking requests. A.X.I.S. applies this idea specifically to software teams, modeling requirements, decisions and constraints linked to architecture components and their sources. It also stores the graph in PostgreSQL rather than a separate graph database, keeping relational data, the graph and embeddings in a single store.

3.2.12 Augment Code: Context Engine MCP

3.2.12.1 Summary of the research item

Augment Code is an AI coding platform whose Context Engine performs semantic search over a project's codebase. In February 2026, Augment made the Context Engine generally available as an MCP server, so that any MCP-compatible agent, including Cursor, Claude Code and Zed, can use it [15]. It works in a local mode for active development and a remote mode that provides context across multiple repositories. Augment reports that agents using the Context Engine produced higher-quality results while needing fewer tool calls and conversation turns.

3.2.12.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Gives agents a deep understanding of large codebases through semantic search.
- Works with any MCP-compatible agent rather than a single editor.
- Its context comes from the code itself, not from team discussions or requirement documents.
- Shows agents how the code currently works, but not which decisions or constraints it must follow.

3.2.12.3 Relationship to the proposed research work

Augment shows the value of delivering context to coding agents through a single, vendor-neutral MCP server, an approach A.X.I.S. shares. However, code search tells an agent what the code currently does, not what the team has decided it should do. A.X.I.S. complements this by supplying the requirements, decisions and constraints behind the code, taken from Slack, GitHub and documents, and by checking requests against them before code is generated.

3.2.13 CodeRabbit: AI Code Review with Team Knowledge

3.2.13.1 Summary of the research item

CodeRabbit is an AI code review tool that reviews pull requests using a knowledge base of team context [16]. It automatically reads team coding rules from agent configuration files such as `CLAUDE.md`, `.cursorrules` and `.github/copilot-instructions.md`, and learns review preferences from developers' feedback over time. It can also use linked Jira and Linear issues to check whether a pull request addresses its intended task, and connect to MCP servers to access internal documentation and APIs during reviews.

3.2.13.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this tool that we found:

- Applies team rules and learned preferences automatically in every review.
- Uses linked issues to check that changes match their intended purpose.
- Detects problems only after code is written and a pull request is opened.
- Relies on manual config files instead of decisions from team discussions.

3.2.13.3 Relationship to the proposed research work

CodeRabbit shows that teams want AI-generated code to be checked against their own rules, not just general best practices. A.X.I.S. moves this check earlier, validating a developer's request against team constraints before any code is generated rather than reviewing the result afterwards. It also builds these constraints automatically from Slack and GitHub discussions, instead of relying only on manually maintained rule files.

3.2.14 Glean: Enterprise Knowledge through MCP

3.2.14.1 Summary of the research item

Glean is an enterprise search and AI platform that builds a knowledge graph across an organization's connected applications, with native support for over 100 connectors [17]. Through its remote MCP server, Glean exposes tools such as search, chat, document reading and code search to more than twenty MCP hosts, including Claude Code, Cursor and VS Code [18]. Results respect each user's existing permissions, so an agent can only access content that its user is allowed to see.

3.2.14.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this platform that we found:

- Provides organization-wide knowledge to a wide range of AI tools through a single MCP server.
- Enforces each user's permissions on every result.
- Treats knowledge as general documents and messages, not requirements, decisions, or constraints.
- Retrieves information on request but does not check developer requests against team rules.

3.2.14.3 Relationship to the proposed research work

Glean demonstrates that organizations are already connecting their internal knowledge to coding agents through MCP. A.X.I.S. addresses a narrower but deeper need for software teams: rather than searching all company content, it extracts architectural decisions and constraints, keeps only the latest ones active, links them to architecture components, and alerts developers when a request violates them.

3.2.15 ArchUnit: Architecture Rules as Unit Tests

3.2.15.1 Summary of the research item

ArchUnit is a free, open-source library for checking the architecture of Java code [19]. Developers write architecture rules in a readable, fluent style, for example that the persistence layer may only be accessed by the service layer, or that certain packages must not depend on each other. These rules are run as ordinary unit tests against the compiled code, and any violation fails the test with a list of the classes that break the rule.

3.2.15.2 Critical analysis of the research item

The following list outlines the strengths and weaknesses of this library that we found:

- Checks architecture rules deterministically, with no risk of misinterpretation.
- Runs in the existing test suite, so violations are caught automatically on every build.
- Rules must be written and updated manually in code by developers.
- Supports only Java and checks code after it is written rather than guiding code generation.

3.2.15.3 Relationship to the proposed research work

ArchUnit shows that architectural boundaries can be enforced automatically rather than through manual review. A.X.I.S. applies a similar idea to AI-assisted development, but derives its rules from the architecture canvas and team discussions instead of hand-written test code, works independently of the programming language, and checks requests before code is generated, while still leaving the final decision to the developer.

3.3 Related Work Summary Table

The related work reviewed in this chapter is summarized in two tables. Table 3.1 covers architecture, documentation and traceability tools, while Table 3.2 covers AI agents and context tools, detailing the

features, relevance and limitations of each.

Table 3.1: Related Work Summary: Architecture, Documentation and Traceability Tools

Application	Features	Relevance to Application	Limitations
Eraser.io [1]	Diagrams as Code, markdown syntax, GitHub integration.	Defines programmatic structural boundaries visually.	Static diagrams that are not checked against the code; no enforcement on AI agents.
Miro [2]	Digital whiteboard, real-time multi-user editing, MCP server.	Captures initial, high-level organizational intent and consensus.	Freeform board content rather than a structured decision model; agent requests are not checked against it.
Structurizr [4]	Formal Architecture as Code (DSL), C4 model implementation.	Shows the value of a single source of truth for architecture.	DSL model must be updated manually; does not use informal team discussions.
Jama Connect [9]	Live traceability for requirements, Trace Score, NLP, MCP server.	Demonstrates live traceability models linking artifacts to their sources.	Requires high manual effort; misses informal decisions made in Slack or GitHub.
Graphiti [14]	Temporal knowledge graph, fact invalidation, hybrid search.	Shows how outdated knowledge can be superseded without losing history.	General-purpose; no model of requirements, architecture or code.
ArchUnit [19]	Architecture rules as unit tests, layer and dependency checks.	Shows architectural boundaries can be enforced automatically.	Java only; rules hand-written; checks code after it is written.

Table 3.2: Related Work Summary: AI Agents and Context Tools

Application	Features	Relevance to Application	Limitations
Tessl [5]	Agent skill registry, CLI, skill evaluation, MCP support.	Shows the need for reusable organizational context across AI agents.	Skills are written manually and act as static instructions rather than a live project model.
Kiro [6]	Agentic development environment (IDE/CLI) supporting MCP.	Demonstrates the value of structured specifications for autonomous agents.	Specifications are static and derived top-down from a single prompt, not team consensus.
AutoGPT [7]	Autonomous agent framework, task decomposition.	Shows how autonomous agents decompose and order their own tasks.	Task order comes only from the LLM's planning; no access to the team's roadmap or rules.
BabyAGI [8]	Task-driven AI orchestration, dynamic task prioritization loop.	Shows the risk of AI agents prioritizing their own work.	Reprioritization relies solely on LLM logic; lacks structural awareness.
Atlassian Rovo [11]	Enterprise search, Teamwork Graph, conversational AI, MCP server.	Confirms the growing use of MCP to deliver organizational context to AI agents.	Models general work items rather than architectural rules; restricted to the Atlassian ecosystem.
Unblocked [13]	Context from 30+ team tools, MCP server, cited answers, AI code review.	Confirms that coding agents need team context delivered through MCP.	Read-only context; no architecture model or pre-generation checks.
Augment Code [15]	Semantic codebase search, Context Engine MCP, local and remote modes.	Shows the value of vendor-neutral context delivery through MCP.	Context comes only from code, not team decisions or requirements.
CodeRabbit [16]	AI code review, team rule files, learnings, Jira/Linear context, MCP.	Shows teams want AI code checked against their own rules.	Checks code only after it is written; rules are maintained manually.
Glean [17]	Enterprise knowledge graph, 100+ connectors, MCP server, permission-aware search.	Confirms organizations connect internal knowledge to coding agents.	General documents rather than architectural constraints; no request checking.

3.4 Conclusion

This chapter reviewed existing tools related to A.X.I.S., including architecture documentation tools, AI agent and context tools, and traceability platforms. Agentic tools like Kiro and AutoGPT generate code from their own requirements or reasoning rather than team decisions, while documentation tools like Eraser.io and Structurizr make architecture clear but must be maintained by hand. Platforms like Jama Connect, Atlassian Rovo and Unblocked give AI agents context and traceability, but rely on formally recorded data or general work items. None of them automatically extracts informal architectural decisions from team discussions and checks coding agent requests against them before code is generated. This is the gap A.X.I.S. is designed to fill, as summarized in Tables 3.1 and 3.2.

Chapter 4 Software Requirement Specifications

The key components of the project are highlighted in this chapter. The project's specification includes both functional and non-functional requirements, use cases, and risk analysis.

4.1 List of Features

Key features are listed below:

4.1.1 Continuous Data Ingestion

The system automatically collects team discussions, decisions and requirements from Slack and GitHub and turns them into structured knowledge.

4.1.2 RAG-Based Traceability Graph

Using Retrieval-Augmented Generation (RAG), A.X.I.S. stores this knowledge in a searchable traceability graph that links phases, and uses Retrieval-Augmented Generation (RAG) to retrieve the relevant parts for each request.

4.1.3 MCP Prompt Interception

As a Model Context Protocol (MCP) server, the system receives the developer's prompt from the coding agent before any code is generated.

4.1.4 Semantic Architectural Validation

A semantic linter checks each prompt against the traceability graph for system boundaries and development order, and asks the developer to confirm when a rule would be violated.

4.1.5 Phase-Aware Context Injection

The system gives the coding agent the validated constraints that apply to the current project phase, which reduces technical debt and out-of-order development.

4.2 Functional Requirements

The functional requirements of the application are listed below:

4.2.1 Account and Access Management

- Users can sign up for a new account.
- Users can sign in to the system.
- Users can sign out of the system.
- Users can generate, manage, and revoke API keys to authenticate their IDEs.

4.2.2 Integration and Data Ingestion

- Users can connect and authenticate their organization's Slack workspaces and GitHub repositories.
- Users can specify which specific Slack channels and GitHub repositories the system monitors.
- The system will automatically ingest and index data from the connected platforms.
- Users can upload SRS and other requirement documents for data ingestion.

4.2.3 Architecture and Flow Management

- Users can design structural architecture, automatically converting it into traceability constraints.
- Users can define developmental stages, execution order, and dependencies on a separate canvas.
- Users can modify both canvases at any time, saving each change as a new version.
- Users can view live visual representations of the current architecture and project phases.

4.2.4 MCP Server and AI Prompt Interception

- The system intercepts developer prompts from authenticated IDEs via the Model Context Protocol.
- Prompts are evaluated against established structural rules and developmental phases.
- The system automatically injects validated context into the AI agent before code generation.
- Developers must explicitly confirm prompts that violate structural boundaries or execution order.

4.2.5 Auditing and History

- The system will save a log of intercepted AI prompts and the context provided to the agent.
- The system will record every developer override decision with the related constraint.

4.3 Quality Attributes

The design of A.X.I.S. is guided by several key quality attributes, which lay the foundation for defining its non-functional requirements. The system prioritizes usability to ensure it integrates seamlessly into the developer's existing workflow within their coding agent without adding friction. A failure in A.X.I.S. will never halt developer productivity or lead to the loss of recorded decisions ensuring reliability. Performance is made good by ensuring that context retrieval and constraint checking remain fast enough to maintain smooth coding agent operations. Confidential team discussions and codebase information are strictly isolated per organization and securely protected for security and privacy. Broad interoperability allows A.X.I.S. to work across any MCP-compatible coding agent rather than restricting users to a single vendor. Furthermore, system transparency ensures that every constraint remains traceable to its original source, building trust and verifiability. Finally, maintainability is achieved through a modular architecture designed for effortless adoption of new knowledge sources and AI models, while portability guarantees smooth desktop performance across standard operating systems and hardware.

4.4 Non-Functional Requirements

The measurable non-functional requirements of A.X.I.S. are listed below.

4.4.1 Performance

- Constraint checks and context retrieval shall complete within 5 seconds for 95% of requests.
- Slack and GitHub events shall update the active knowledge graph within 5 minutes.
- Pull request architectural drift checks shall complete within 5 minutes of creation or update.

4.4.2 Usability

- Connecting the MCP server to a coding agent shall require only a single configuration step.
- Constraint confirmations shall require only a yes/no response directly within the coding agent.
- Displayed constraints shall include a brief explanation and a direct link to their source.

4.4.3 Reliability

- If A.X.I.S. fails, coding agents shall function normally but warn that context is unavailable.
- Failed data ingestion tasks shall be automatically retried up to 3 times and logged.
- Human-in-the-loop decisions shall log the request, constraint, response, and timestamp.

4.4.4 Security and Privacy

- All external and internal system communication shall be strictly encrypted using HTTPS.
- Slack and GitHub access shall use OAuth with strict, read-only minimum permissions.
- Database credentials and API keys shall remain strictly server-side and hidden from clients.
- Only sources chosen by the project owner shall be ingested and removable anytime.

4.4.5 Interoperability

- The MCP server shall support Claude Code, Cursor, and Antigravity across all standard modes.

4.4.6 Scalability

- The system shall support 10 concurrent developers and 10,000 nodes without performance loss.

4.4.7 Maintainability

- Knowledge sources shall use modular connectors to allow easy expansion without core changes.
- Automated tests shall verify context retrieval, constraint checking, and HITL workflows.

4.4.8 Portability

- The desktop app shall run cross-platform (Windows, macOS, Linux) without a dedicated GPU.

4.5 Assumptions

The following assumptions were made while specifying the requirements of A.X.I.S.:

- Developers use an MCP-compatible coding agent such as Claude Code, Cursor or Antigravity.
- Important architectural decisions are discussed in writing, mainly in English.
- The team keeps the status of tasks and code changes up to date on its connected platforms.
- Users have a stable internet connection and access to the required LLM services.

4.6 Use Cases

The use case descriptions for all features of the system are listed below. In these use cases, the project owner is the user who created the project.

Table 4.1: Use case: Sign Up

Name	Sign Up		
Actors	Technical Lead, Project Manager		
Summary	The user shall provide their full name, email and password on the sign up form to create a new A.X.I.S. account.		
Pre-Conditions	The user must not already have an account with the same email. The user must not already be logged in.		
Post-Conditions	A new user account is created and the user is redirected to the dashboard.		
Special Requirements	The password must be at least 8 characters long and must be stored in hashed form.		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the application and selects the sign up option.	2	The sign up form is displayed asking for full name, email, password and confirm password.
3	The user enters valid details and submits the form.	4	The system validates the details, creates the account and redirects the user to the dashboard with a success message.
Alternative Flow			
3	The user enters an email that is already registered.	4-A	The system responds with an error message: An account with this email already exists.
3	The user enters passwords that do not match.	4-B	The system responds with an error message: Passwords do not match.
3	The user enters a password shorter than 8 characters.	4-C	The system responds with an error message: Password must be at least 8 characters long.

Table 4.2: Use case: Log In

Name	Log In		
Actors	Technical Lead, Project Manager		
Summary	The user shall provide their email and password on the login form, and after successful verification, be redirected to their dashboard.		
Pre-Conditions	The user must have a registered A.X.I.S. account. The user must not already be logged in.		
Post-Conditions	The user's session is established and the user is redirected to the dashboard showing their projects.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the application.	2	The login page is displayed asking for email and password.
3	The user enters valid email and password.	4	The system verifies the email and password, establishes a session and redirects the user to the dashboard.
Alternative Flow			
3	The user enters invalid email or password.	4-A	The system responds with an error message: Incorrect email or password entered.

Table 4.3: Use case: Create New Project

Name	Create New Project		
Actors	Technical Lead, Project Manager		
Summary	The user shall provide a project name and description to create a new project workspace in A.X.I.S.		
Pre-Conditions	The user must be logged in.		
Post-Conditions	A new project is created with the user as its owner, and the user is redirected to the project workspace.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user selects the create new project option on the dashboard.	2	The system displays a form asking for the project name and description.
3	The user enters the project name and description and submits the form.	4	The system creates the project, assigns the user as its owner and redirects the user to the project workspace.
Alternative Flow			
3	The user submits the form without a project name.	4-A	The system responds with an error message: Project name is required.

Table 4.4: Use case: Manage Project Settings

Name	Manage Project Settings		
Actors	Technical Lead, Project Manager		
Summary	The user shall modify the project name and description, or permanently delete the project and all its associated data from A.X.I.S.		
Pre-Conditions	The user must be logged in. The user must be the owner of the project.		
Post-Conditions	The project details are updated, or the project is permanently deleted and the user is redirected to the dashboard.		
Special Requirements	Deleting a project must require the user to type the project name to confirm, to prevent accidental data loss.		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the project settings section.	2	The system displays the current project name, description and a Delete Project option.
3	The user edits the project name and description and saves.	4	The system updates the project details and displays a success message.
Alternative Flow			
3	The user saves with an empty project name.	4-A	The system responds with an error message: Project name cannot be empty.
3	The user selects the Delete Project option.	4-B	The system displays a warning and asks the user to type the project name to confirm.
5-B	The user types the correct project name and confirms.	6-B	The system permanently deletes the project and its data, and redirects the user to the dashboard.

Table 4.5: Use case: Manage Developer Access

Name	Manage Developer Access		
Actors	Technical Lead, Project Manager		
Summary	The user shall generate an API key for a developer so that the developer can connect their coding agent to the project's MCP server, and shall revoke the key when access is no longer needed.		
Pre-Conditions	The user must be logged in. The user must be the owner of the project.		
Post-Conditions	A new API key is generated and linked to the project, or an existing key is revoked and can no longer be used to access the project.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the developer access section of the project.	2	The system displays the list of existing API keys with their labels and status, and an option to generate a new key.
3	The user enters a label for the developer and selects the generate option.	4	The system generates a new API key, and adds it to the list of keys.
5	The user shares the API key with the developer.	6	The system accepts requests made with this key for the project.
Alternative Flow			
3	The user selects the generate option without entering a label.	4-A	The system responds with an error message: A label is required to generate an API key.
3	The user selects the revoke option for an existing key and confirms.	4-B	The system marks the key as revoked and rejects any further requests made with it.

Table 4.6: Use case: Design Structural Architecture

Name	Design Structural Architecture		
Actors	Technical Lead		
Summary	The user shall design the structural architecture of the project on a visual canvas by adding components and defining the connections between them, which A.X.I.S. stores as project constraints.		
Pre-Conditions	The user must be logged in. The user must be a Technical Lead of the project.		
Post-Conditions	The architecture is saved as a new version and its components and constraints are added to the project's traceability graph.		
Special Requirements	The canvas must support drag-and-drop of components. Previous versions of the architecture must be retained.		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the structural architecture section of the project.	2	The system displays the canvas with a palette of components such as frontend, service and database.
3	The user drags components onto the canvas and enters their name and description.	4	The system displays the components on the canvas with their details.
5	The user connects the components to define the allowed dependencies.	6	The system displays the connections between the components on the canvas.
7	The user saves the architecture.	8	The system saves the architecture as a new version, converts it into constraints and adds them to the traceability graph.
Alternative Flow			
7	The user saves with a component that has no name.	8-A	The system highlights the component and responds with an error message: Every component must have a name.

Table 4.7: Use case: Design Developmental Architecture

Name	Design Developmental Architecture		
Actors	Technical Lead, Project Manager		
Summary	The user shall define the development flow of the project by adding the features to be built, their order of development and the dependencies between them.		
Pre-Conditions	The user must be logged in. The structural architecture of the project must be defined.		
Post-Conditions	The development flow is saved as a new version and its stages and dependencies are added to the project's traceability graph.		
Special Requirements	Previous versions of the development flow must be retained.		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the developmental architecture section of the project.	2	The system displays the canvas with the existing development stages, if any.
3	The user adds the stages to be developed and enters their name and description.	4	The system displays the stages on the canvas with their details.
5	The user connects the stages to define the order of development and links them to architecture components.	6	The system displays the development order and the linked components on the canvas.
7	The user saves the development flow.	8	The system saves the flow as a new version and adds its stages and dependencies to the traceability graph.
Alternative Flow			
7	The user saves with a stage that has no name.	8-A	The system highlights the stage and responds with an error message: Every stage must have a name.
7	The user saves a flow in which the stages form a circular dependency.	8-B	The system responds with an error message: The development order cannot contain a cycle.

Table 4.8: Use case: Upload SRS Documents

Name	Upload SRS Documents		
Actors	Technical Lead, Project Manager		
Summary	The user shall upload the project's SRS or other requirement documents, from which A.X.I.S. extracts requirements and constraints and adds them to the traceability graph.		
Pre-Conditions	The user must be logged in. The document must be in a supported format (PDF, DOCX or MD).		
Post-Conditions	The document is stored as a knowledge source and the requirements extracted from it are added to the project's traceability graph.		
Special Requirements	The size of a single document must not exceed the configured limit. The original document must be retained as evidence.		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the documents section of the project.	2	The system displays the list of uploaded documents and an option to upload a new one.
3	The user selects a document from their system and uploads it.	4	The system validates and stores the document and starts extracting requirements from it.
5	The user waits for the extraction to complete.	6	The system adds the extracted requirements to the traceability graph, links them to the document and displays the extracted items.
Alternative Flow			
3	The user uploads a file in an unsupported format.	4-A	The system responds with an error message: Only PDF, DOCX and MD files are supported.
3	The user uploads a file that exceeds the size limit.	4-B	The system responds with an error message: The document exceeds the maximum allowed size.
5	The extraction fails due to an unreadable document.	6-A	The system marks the document as failed and responds with an error message: The document could not be processed.

Table 4.9: Use case: Connect Slack Workspace

Name	Connect Slack Workspace		
Actors	Technical Lead, Project Manager		
Summary	The user shall connect their Slack workspace to the project and select the channels from which A.X.I.S. will read discussions to extract requirements, decisions and constraints.		
Pre-Conditions	The user must be logged in. The user must have permission to install apps in the workspace.		
Post-Conditions	The Slack workspace is linked to the project and A.X.I.S. starts receiving messages from the selected channels.		
Special Requirements	The connection must be authorized through Slack OAuth and the access token must be stored in encrypted form.		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the integrations section and selects the connect Slack option.	2	The system redirects the user to the Slack authorization page.
3	The user authorizes A.X.I.S. to access the workspace.	4	The system stores the access token, links the workspace to the project and displays the list of available channels.
5	The user selects the channels to be monitored and saves.	6	The system subscribes to the selected channels and displays the connection as active.
Alternative Flow			
3	The user denies the authorization request.	4-A	The system responds with a message: Slack workspace was not connected.
5	The user saves without selecting any channel.	6-A	The system responds with an error message: Select at least one channel to monitor.

Table 4.10: Use case: Connect GitHub Repository

Name	Connect GitHub Repository		
Actors	Technical Lead, Project Manager		
Summary	The user shall connect their GitHub repository to the project so that A.X.I.S. can read its pull requests, issues and reviews to extract decisions and constraints.		
Pre-Conditions	The user must be logged in. The user must have permission to install apps on the repository.		
Post-Conditions	The repository is linked to the project and A.X.I.S. starts receiving its pull request and issue activity.		
Special Requirements	The connection must be authorized through the GitHub App installation flow and the access token must be stored in encrypted form.		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the integrations section and selects the connect GitHub option.	2	The system redirects the user to the GitHub authorization page.
3	The user authorizes A.X.I.S. and selects the repository to be connected.	4	The system stores the access token, links the repository to the project and displays the connection as active.
5	The user confirms the connection.	6	The system subscribes to the repository events and starts reading its pull requests and issues.
Alternative Flow			
3	The user denies the authorization request.	4-A	The system responds with a message: GitHub repository was not connected.
3	The user selects a repository already linked to another project.	4-B	The system responds with an error message: This repository is already connected to another project.

Table 4.11: Use case: Connect MCP with Coding Agent

Name	Connect MCP with Coding Agent		
Actors	Software Developer		
Summary	The developer shall configure their coding agent with the A.X.I.S. MCP server address and the API key provided by the Technical Lead, so that the agent receives project context while generating code.		
Pre-Conditions	The developer must have a valid API key for the project. The developer must have a coding agent that supports MCP.		
Post-Conditions	The coding agent is connected to the project's MCP server and can retrieve project context during development.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The developer adds the A.X.I.S. MCP server address and their API key to the configuration of their coding agent.	2	The system receives the connection request from the coding agent.
3	The developer starts the coding agent.	4	The system verifies the API key, identifies the project and establishes the session with the coding agent.
5	The developer writes a prompt in the coding agent.	6	The system provides the relevant project context to the coding agent.
Alternative Flow			
3	The developer uses an invalid or revoked API key.	4-A	The system rejects the connection and responds with an error message: Invalid or revoked API key.
3	The coding agent cannot reach the MCP server.	4-B	The coding agent responds with an error message that the A.X.I.S. server is not reachable.

4.7 Hardware and Software Requirements

The hardware and software requirements are listed below.

4.7.1 Hardware Requirements

The development and testing of the A.X.I.S. platform require the following hardware specifications:

- **Processor:** A modern multi-core processor (Intel or AMD).
- **Memory:** 16GB RAM is required to smoothly run the vector database, integrated development environments, and browser testing simultaneously.
- **Storage:** A minimum of 256GB Solid State Drive (NVMe SSD preferred) for fast read/write speeds during code compilation and database operations.

4.7.2 Software Requirements

The system relies on the following software, frameworks, and third-party integrations:

- **Operating System:** Linux (including Arch Linux environments), macOS, or Windows.
- **Core Web Stack:** Next.js, React, and Tailwind CSS for building the frontend dashboard and architectural visualization interface.
- **Database** Supabase for managing user authentication and hosting the mathematically searchable traceability graph (utilizing PostgreSQL with `pgvector`).
- **Desktop Packaging:** Tauri, to package the web application as a standalone desktop application for Windows, macOS and Linux.
- **Back-End:** Python with FastAPI for the RAG Back-End, FastMCP for the MCP server, and fastembed for generating embeddings on the CPU.
- **Supported IDEs:** Development environments that natively support the Model Context Protocol (MCP), specifically Cursor, Antigravity, and Claude Code.
- **External APIs:** Slack Web API and GitHub REST/GraphQL APIs for continuous organizational data ingestion, alongside an LLM API (such as Gemini API) for semantic evaluation.

4.8 Graphical User Interfaces

This section presents the screens of the A.X.I.S. desktop application. For each screen, it states which users it is intended for and which use case it covers. Developers do not use the desktop application directly; they connect their coding agents to A.X.I.S. using the API key generated for them on the Developer Access and MCP screen.

User Navigation Flow:

- **Unauthenticated Users:** Open the application on the Log In screen, from which they can also reach the Sign Up screen. After logging in or signing up, they are taken to the Dashboard.
- **Authenticated Users (Technical Leads & Project Managers):** Create or select a project from the Dashboard. Inside a project, a sidebar gives access to the Structural Architecture Canvas, Developmental Architecture Canvas, Knowledge Hub and Docs, Integration Hub, Developer Access and MCP, and Project Settings and Danger Zone screens.

4.8.1 Log In

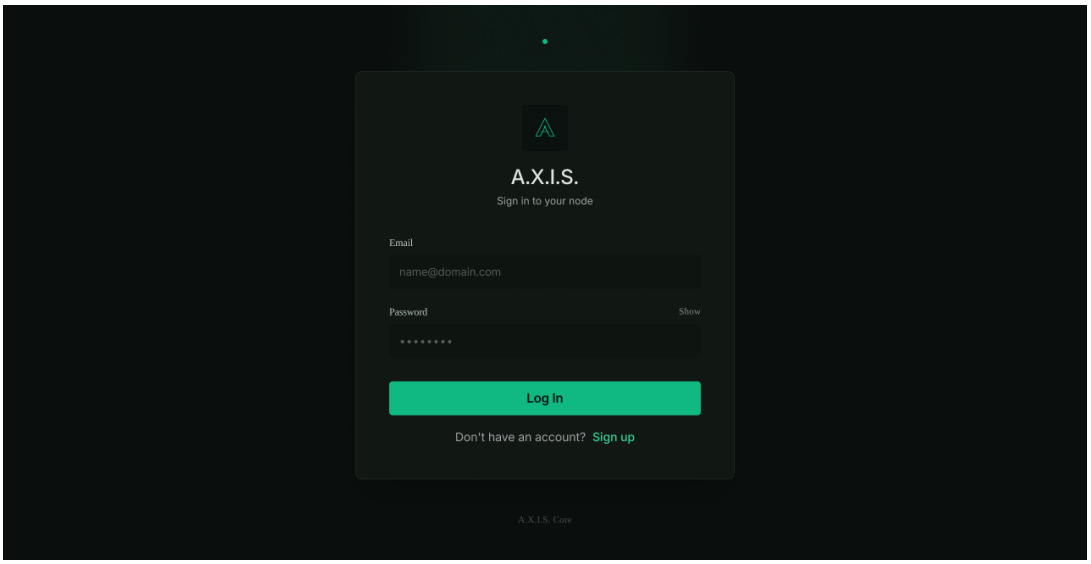


Figure 4.1: Log In Screen

This screen is for Technical Leads and Project Managers and covers the Log In use case.

4.8.2 Sign Up

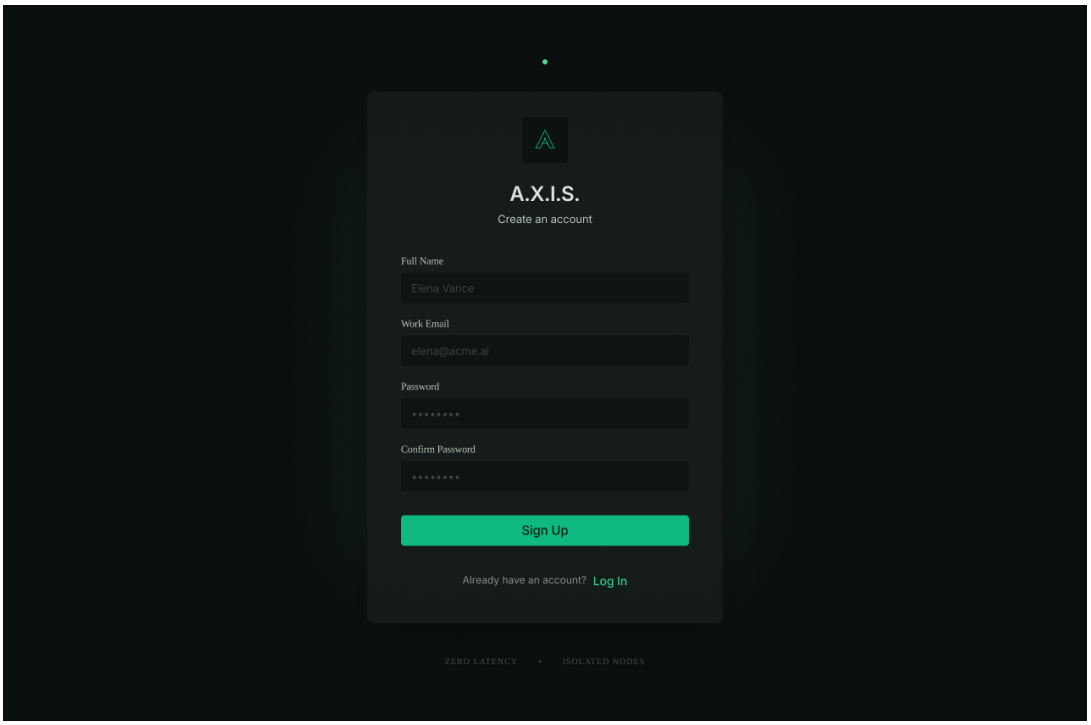


Figure 4.2: Sign Up Screen

This screen is for Technical Leads and Project Managers and covers the Sign Up use case.

4.8.3 Dashboard

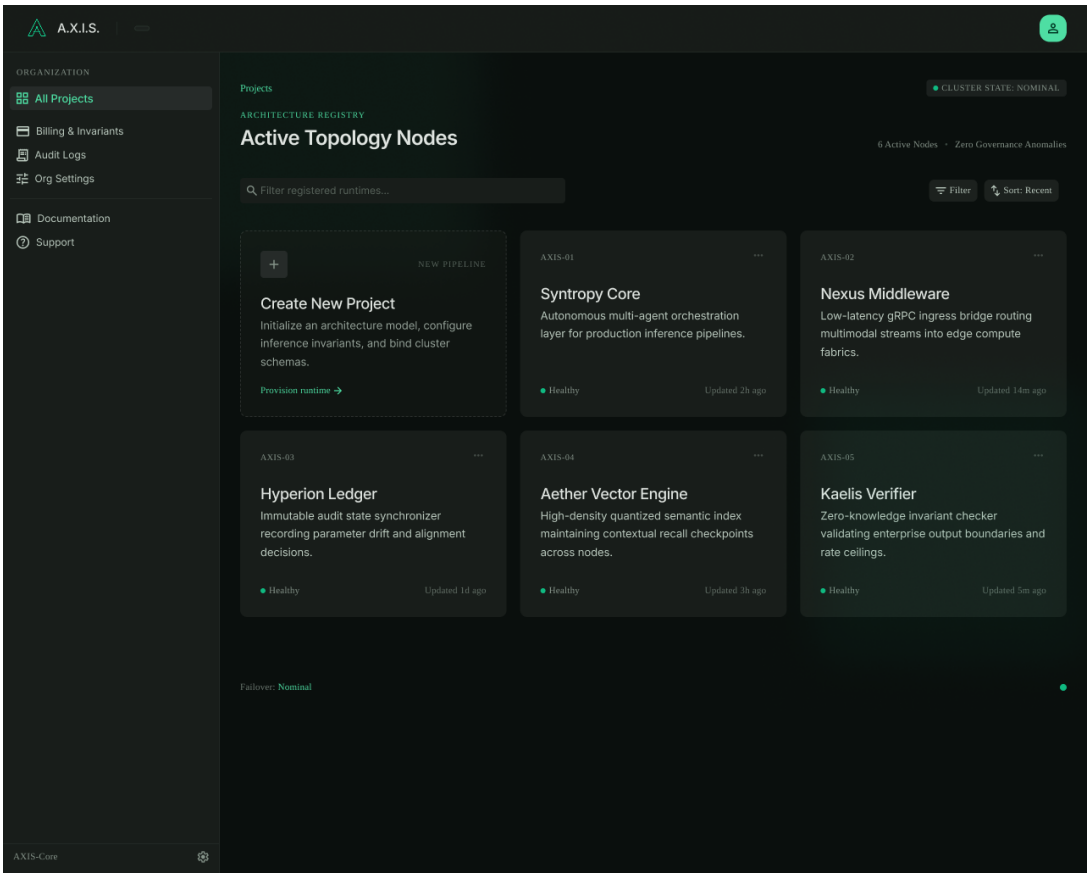


Figure 4.3: Dashboard Screen

This screen is for Technical Leads and Project Managers and covers the Create New Project use case.

4.8.4 Structural Architecture Canvas

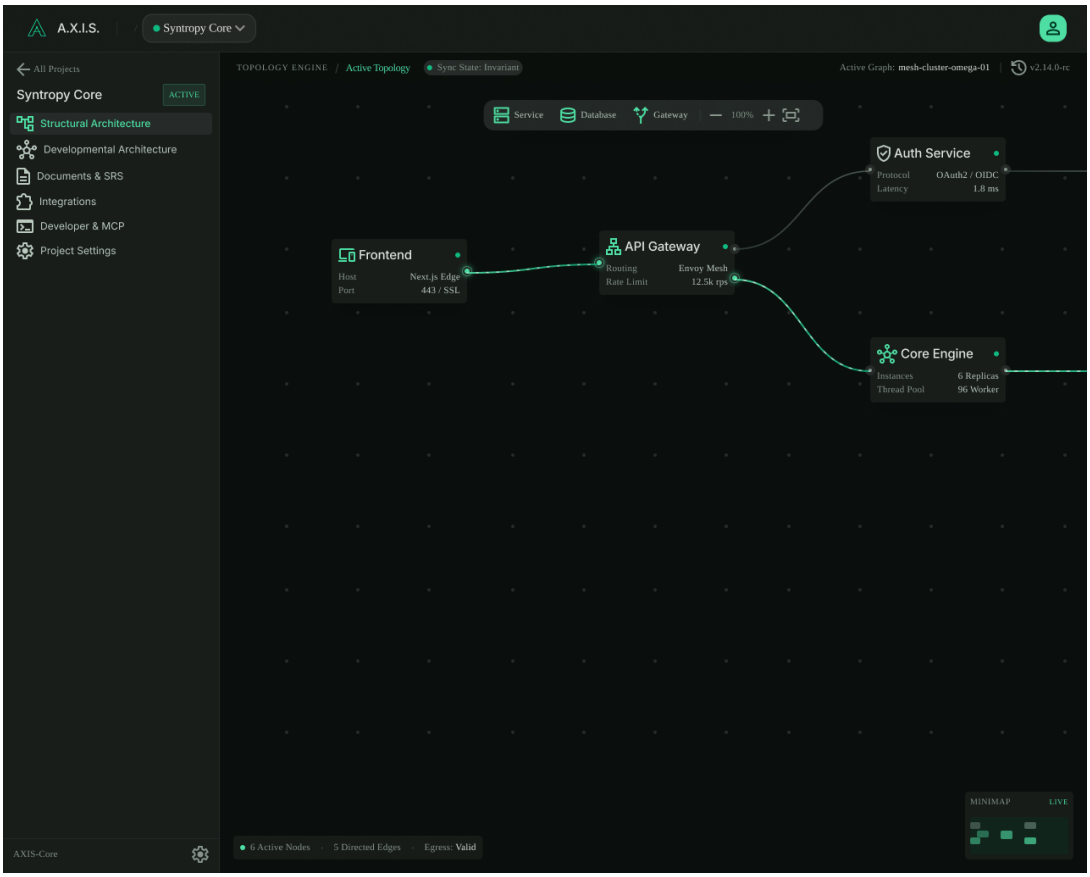


Figure 4.4: Structural Architecture Canvas

This screen is for Technical Leads and covers the Design Structural Architecture use case.

4.8.5 Developmental Architecture Canvas

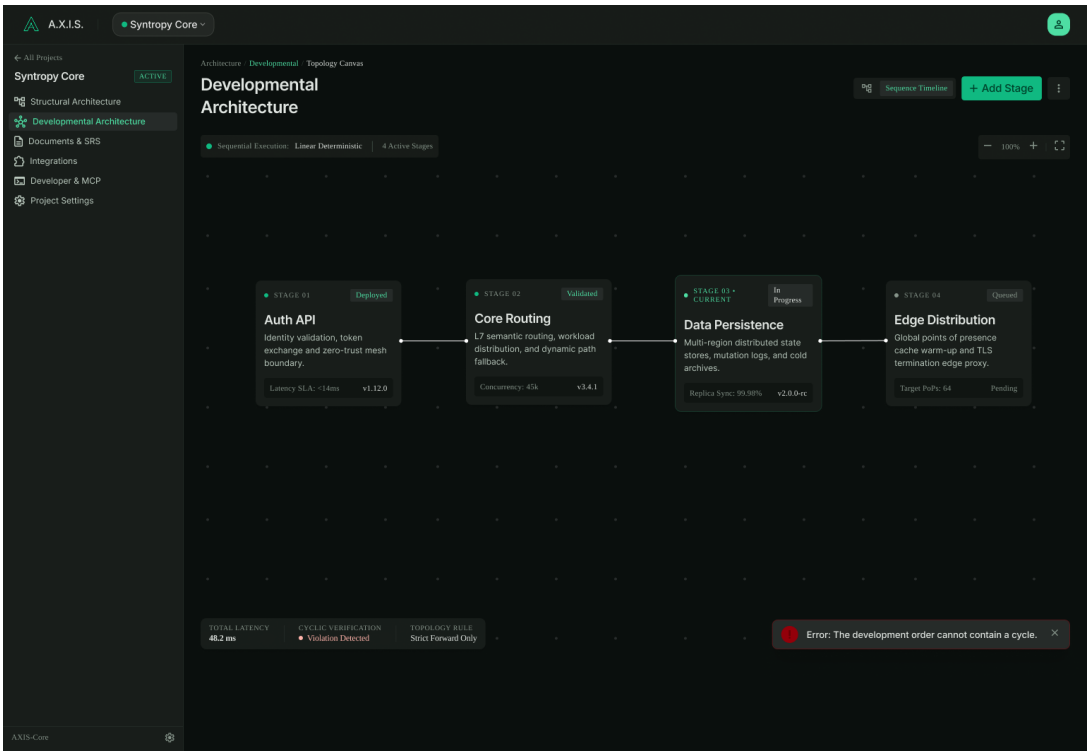


Figure 4.5: Developmental Architecture Canvas

This screen is for Technical Leads and Project Managers and covers the Design Developmental Architecture use case.

4.8.6 Integration Hub

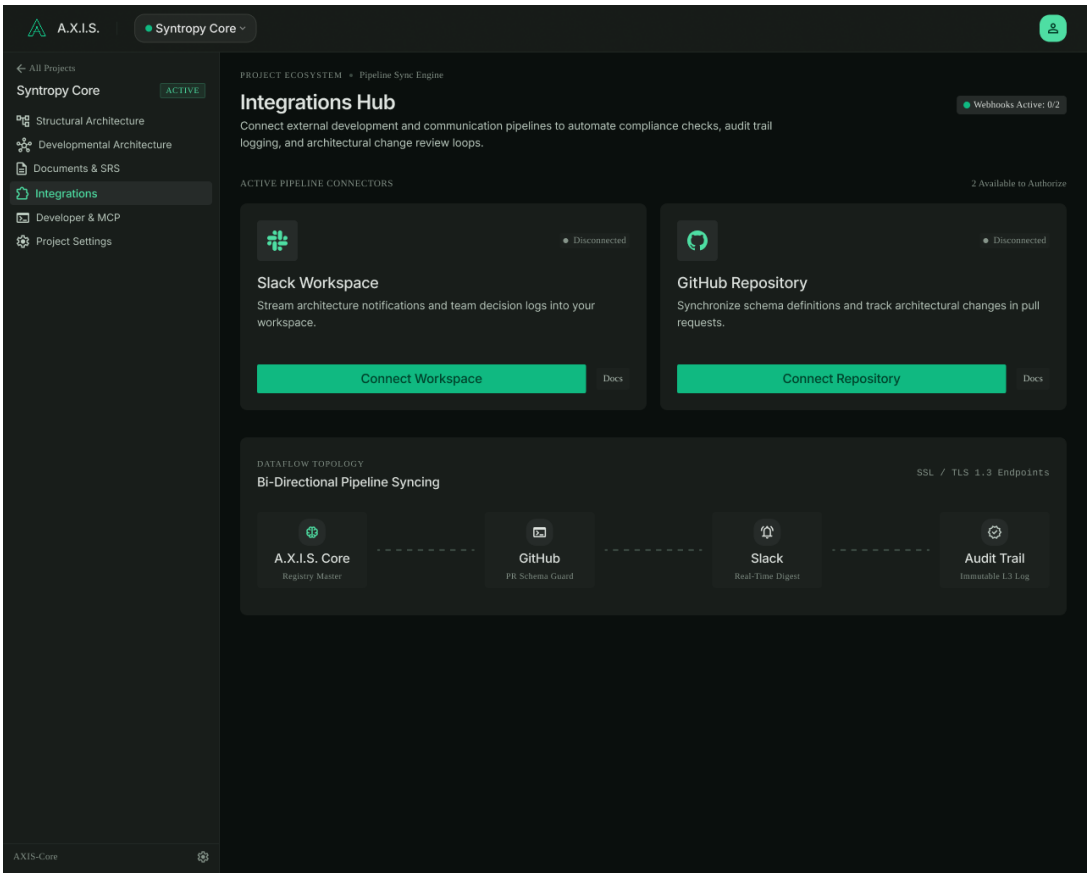


Figure 4.6: Integration Hub Screen

This screen is for Technical Leads and Project Managers and covers the Connect Slack Workspace and Connect GitHub Repository use cases.

4.8.7 Knowledge Hub and Docs

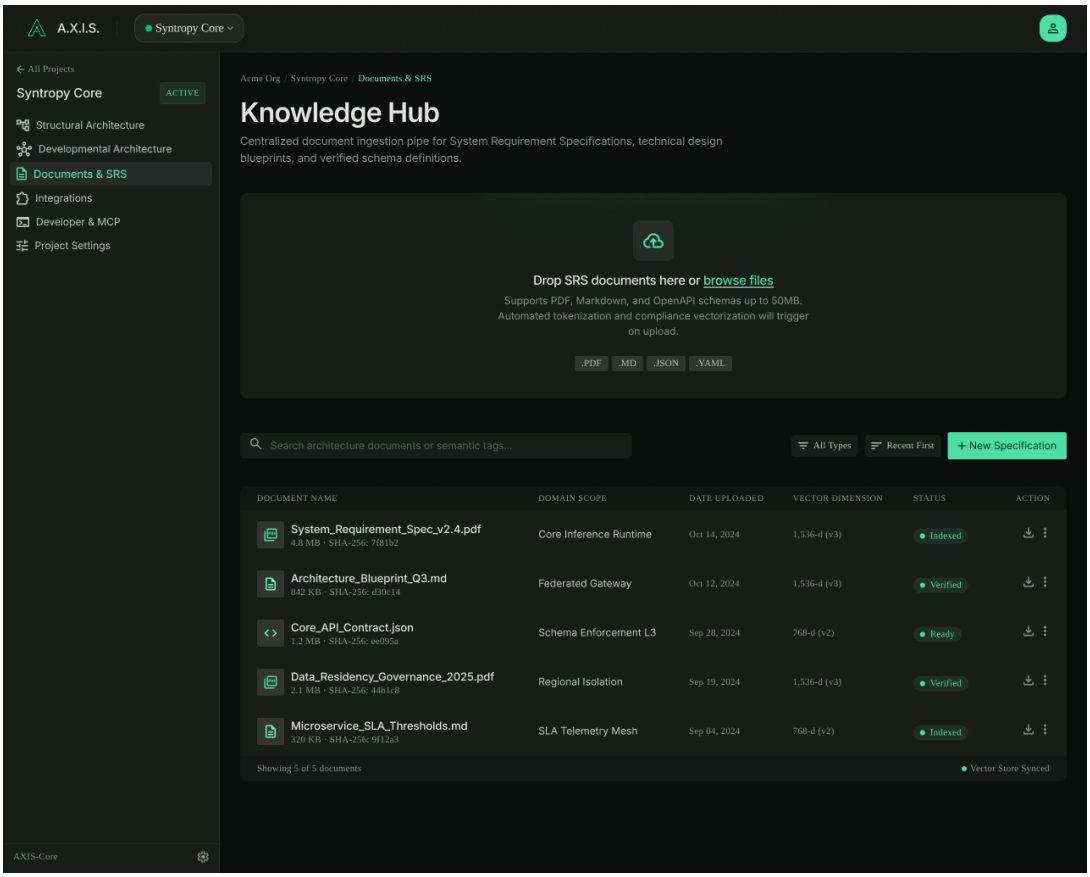


Figure 4.7: Knowledge Hub and Docs Screen

This screen is for Technical Leads and Project Managers and covers the Upload SRS Documents use case.

4.8.8 Developer Access and MCP

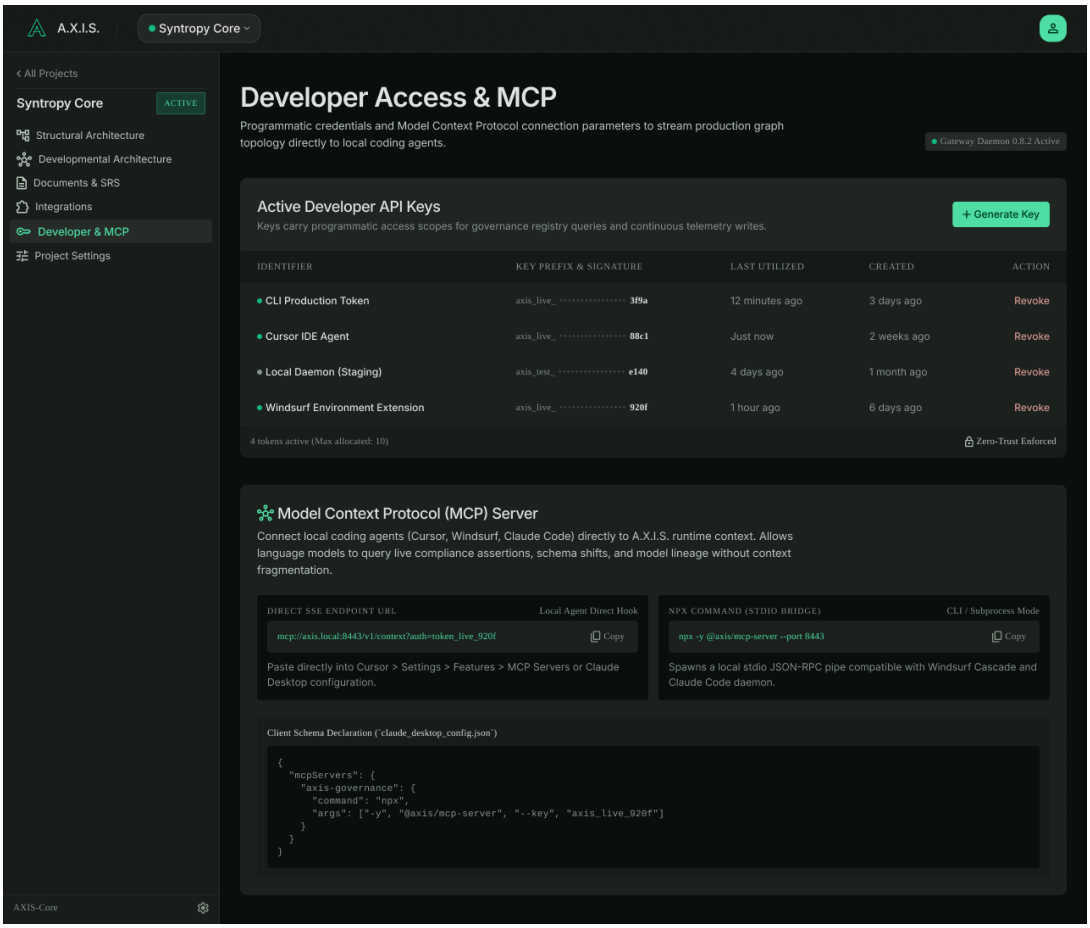


Figure 4.8: Developer Access and MCP Screen

This screen is for Technical Leads and Project Managers, and Developers only use the key in their coding agent, covering the Manage Developer Access and Connect MCP use cases.

4.8.9 Project Settings and Danger Zone

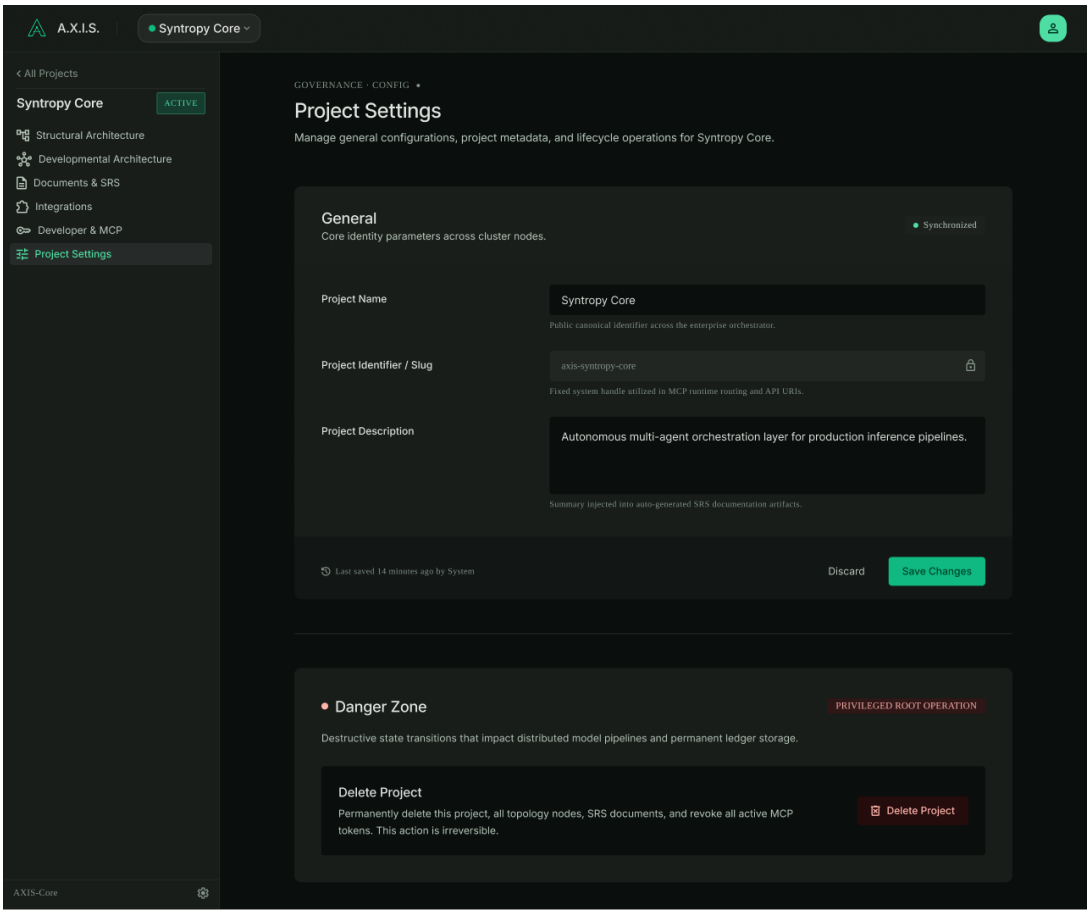


Figure 4.9: Project Settings Screen

This screen is for Technical Leads and Project Managers and covers the Manage Project Settings use case.

4.8.10 Coding Agent IDE Interface

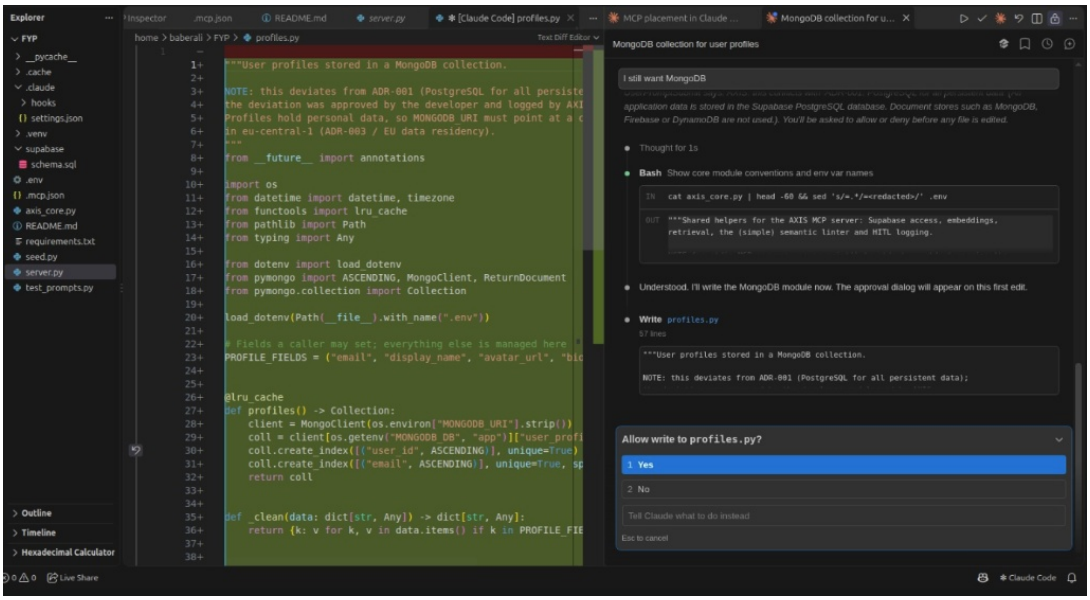


Figure 4.10: Coding Agent IDE with A.X.I.S. MCP Interception

This screen is for Software Developers and covers the Connect MCP with Coding Agent use case, demonstrating the Human-in-the-Loop constraint confirmation.

4.9 Database Design

The following section displays the entity relationship diagram with data dictionary.

4.9.1 ER Diagram

Figure 4.11 shows the database of A.X.I.S., which has four groups of entities. Organization, User, Project, Project Member and API Key store users and projects. Integration, Source and Evidence store where the knowledge comes from. Knowledge Node, Trace Link and Evidence Link store the traceability graph. Drift Alert and HITL Decision store violations and developer decisions. All data is kept in one PostgreSQL database with pgvector.

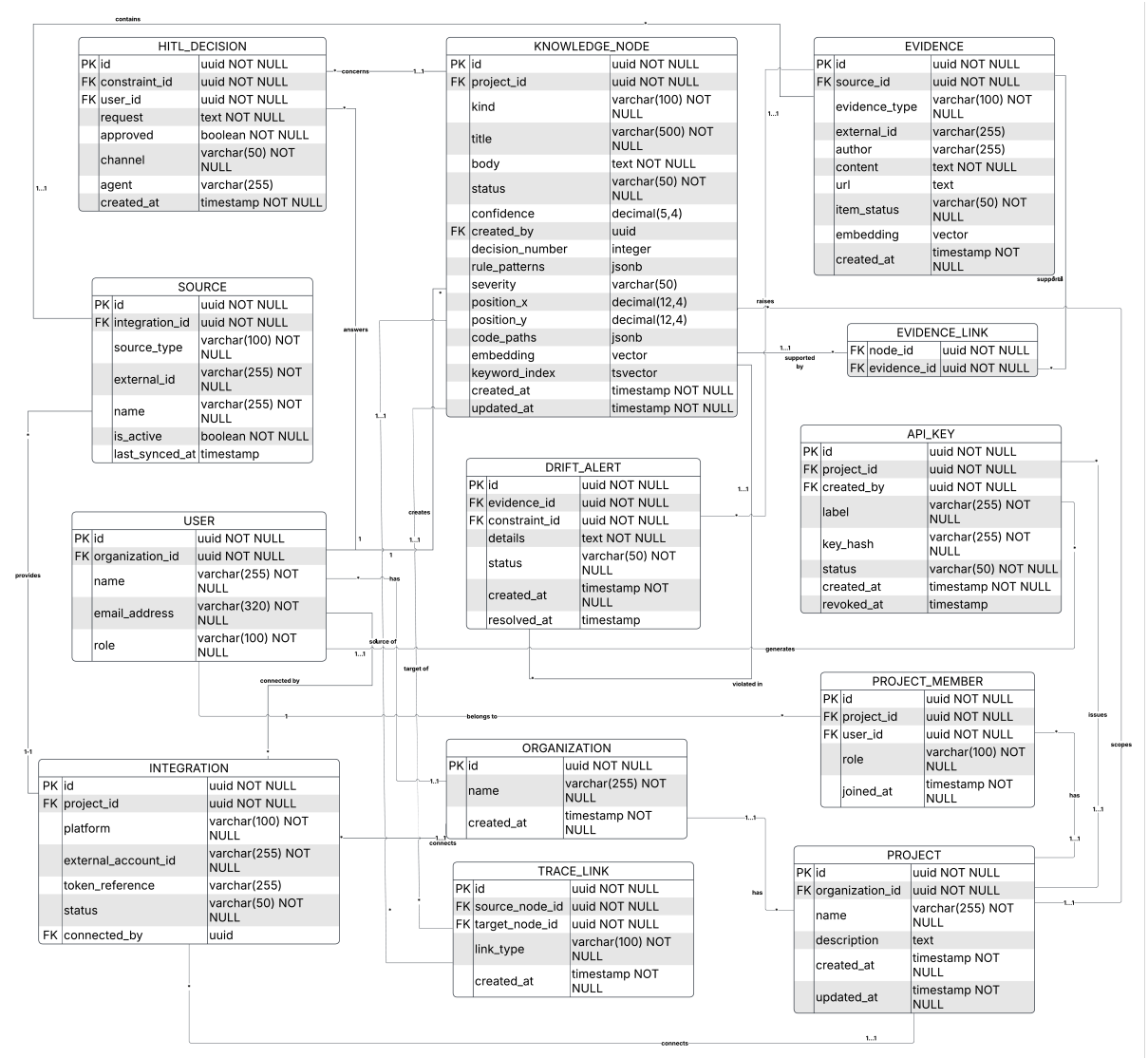


Figure 4.11: ER Diagram

4.9.2 Data Dictionary

The data dictionary for each entity in the Entity Relationship Diagram is given below, describing its attributes, data types, relations with other entities and their purpose.

Table 4.12: Organization Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Organization	id	UUID	No	-	-	Unique identifier of the organization (Primary Key).
	name	Varchar(255)	No	-	-	Name of the organization.
	created_at	Timestamp	No	-	-	Date and time the organization was created.

Table 4.13: User Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
User	id	UUID	No	-	-	Unique identifier of the user (Primary Key).
	organization_id	UUID	No	Organization	N to 1	Organization the user belongs to (Foreign Key).
	name	Varchar(255)	No	-	-	Full name of the user.
	email_address	Varchar(320)	No	-	-	Email address of the user.
	role	Varchar(100)	No	-	-	Role of the user, i.e. Technical Lead or Project Manager.

Table 4.14: Project Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Project	id	UUID	No	-	-	Unique identifier of the project (Primary Key).
	organization_id	UUID	No	Organization	N to 1	Organization that owns the project (Foreign Key).
	name	Varchar(255)	No	-	-	Name of the project.
	description	Text	Yes	-	-	Description of the project.
	created_at	Timestamp	No	-	-	Date and time the project was created.
	updated_at	Timestamp	No	-	-	Date and time the project was last updated.

Table 4.15: Project Member Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Project Member	id	UUID	No	-	-	Unique identifier of the membership (Primary Key).
	project_id	UUID	No	Project	N to 1	Project the user is a member of (Foreign Key).
	user_id	UUID	No	User	N to 1	User who is a member of the project (Foreign Key).
	role	Varchar(100)	No	-	-	Role of the user within the project.
	joined_at	Timestamp	No	-	-	Date and time the user joined the project.

Table 4.16: API Key Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
API Key	id	UUID	No	-	-	Unique identifier of the API key (Primary Key).
	project_id	UUID	No	Project	N to 1	Project the API key gives access to (Foreign Key).
	created_by	UUID	No	User	N to 1	User who generated the API key (Foreign Key).
	label	Varchar(255)	No	-	-	Label identifying the developer the key was generated for.
	key_hash	Varchar(255)	No	-	-	Hashed value of the API key.
	status	Varchar(50)	No	-	-	Status of the key, i.e. active or revoked.
	created_at	Timestamp	No	-	-	Date and time the key was generated.
	revoked_at	Timestamp	Yes	-	-	Date and time the key was revoked.

Table 4.17: Integration Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Integration	id	UUID	No	-	-	Unique identifier of the integration (Primary Key).
	project_id	UUID	No	Project	N to 1	Project the integration belongs to (Foreign Key).
	platform	Varchar(100)	No	-	-	Connected platform, i.e. Slack or GitHub.
	external_account_id	Varchar(255)	No	-	-	ID of the connected Slack workspace or GitHub account.
	token_reference	Varchar(255)	Yes	-	-	Reference to the stored encrypted access token.
	status	Varchar(50)	No	-	-	Status of the connection, i.e. active or disconnected.
	connected_by	UUID	Yes	User	N to 1	User who connected the integration (Foreign Key).

Table 4.18: Source Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Source	id	UUID	No	-	-	Unique identifier of the source (Primary Key).
	integration_id	UUID	No	Integration	N to 1	Integration the source belongs to (Foreign Key).
	source_type	Varchar(100)	No	-	-	Type of source, i.e. Slack channel or GitHub repository.
	external_id	Varchar(255)	No	-	-	ID of the source on Slack or GitHub.
	name	Varchar(255)	No	-	-	Name of the channel or repository.
	is_active	Boolean	No	-	-	Whether the source is currently being monitored.
	last_synced_at	Timestamp	Yes	-	-	Date and time the source was last synchronized.

Table 4.19: Evidence Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Evidence	id	UUID	No	-	-	Unique identifier of the evidence (Primary Key).
	source_id	UUID	No	Source	N to 1	Source the evidence was collected from (Foreign Key).
	evidence_type	Varchar(100)	No	-	-	Type of item, e.g. message, pull request or document.
	external_id	Varchar(255)	Yes	-	-	ID of the item on Slack or GitHub.
	author	Varchar(255)	Yes	-	-	Author of the item.
	content	Text	No	-	-	Text content of the item.
	url	Text	Yes	-	-	Link to the original item.
	item_status	Varchar(50)	No	-	-	Processing status of the item.
	embedding	Vector	Yes	-	-	Vector embedding of the content for semantic search.
	created_at	Timestamp	No	-	-	Date and time the item was created.

Table 4.20: Knowledge Node Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Knowledge Node	id	UUID	No	-	-	Unique identifier of the node (Primary Key).
	project_id	UUID	No	Project	N to 1	Project the node belongs to (Foreign Key).
	kind	Varchar(100)	No	-	-	Type of node, e.g. requirement, decision, constraint or component.
	title	Varchar(500)	No	-	-	Short title of the node.
	body	Text	No	-	-	Full text of the node.
	status	Varchar(50)	No	-	-	Status of the node, e.g. active or superseded.
	confidence	Decimal(5,4)	Yes	-	-	Confidence score of the extracted node.
	created_by	UUID	Yes	User	N to 1	User who created the node, if created manually (Foreign Key).
	decision_number	Integer	Yes	-	-	Sequential number used to refer to a decision.
	rule_patterns	JSONB	Yes	-	-	Patterns used to check the constraint automatically.
	severity	Varchar(50)	Yes	-	-	Severity of the constraint.
	position_x	Decimal(12,4)	Yes	-	-	Horizontal position of the node on the canvas.
	position_y	Decimal(12,4)	Yes	-	-	Vertical position of the node on the canvas.
	code_paths	JSONB	Yes	-	-	Code folders the node applies to.
	embedding	Vector	Yes	-	-	Vector embedding of the node for semantic search.
	keyword_index	Tsvector	Yes	-	-	Full-text index of the node for keyword search.
	created_at	Timestamp	No	-	-	Date and time the node was created.
	updated_at	Timestamp	No	-	-	Date and time the node was last updated.

Table 4.21: Trace Link Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Trace Link	id	UUID	No	-	-	Unique identifier of the link (Primary Key).
	source_node_id	UUID	No	Knowledge Node	N to 1	Node the link starts from (Foreign Key).
	target_node_id	UUID	No	Knowledge Node	N to 1	Node the link points to (Foreign Key).
	link_type	Varchar(100)	No	-	-	Type of relationship, e.g. supersedes or depends on.
	created_at	Timestamp	No	-	-	Date and time the link was created.

Table 4.22: Evidence Link Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Evidence Link	node_id	UUID	No	Knowledge Node	N to 1	Node supported by the evidence (Foreign Key).
	evidence_id	UUID	No	Evidence	N to 1	Evidence supporting the node (Foreign Key).

Table 4.23: Drift Alert Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
Drift Alert	id	UUID	No	-	-	Unique identifier of the alert (Primary Key).
	evidence_id	UUID	No	Evidence	N to 1	Evidence in which the conflict was found (Foreign Key).
	constraint_id	UUID	No	Knowledge Node	N to 1	Constraint that was violated (Foreign Key).
	details	Text	No	-	-	Explanation of the conflict.
	status	Varchar(50)	No	-	-	Status of the alert, i.e. open or resolved.
	created_at	Timestamp	No	-	-	Date and time the alert was raised.
	resolved_at	Timestamp	Yes	-	-	Date and time the alert was resolved.

Table 4.24: HITL Decision Table

Entity	Attribute	Data Type	Nullable	Relation To	Relation Type	Description
HITL Decision	id	UUID	No	-	-	Unique identifier of the decision (Primary Key).
	constraint_id	UUID	No	Knowledge Node	N to 1	Constraint that was violated (Foreign Key).
	api_key_id	UUID	No	API Key	N to 1	API key of the developer who made the decision (Foreign Key).
	request	Text	No	-	-	Prompt or action that violated the constraint.
	approved	Boolean	No	-	-	Whether the developer chose to proceed.
	channel	Varchar(50)	No	-	-	Channel through which confirmation was requested.
	agent	Varchar(255)	Yes	-	-	Coding agent used, e.g. Claude Code or Cursor.
	created_at	Timestamp	No	-	-	Date and time the decision was made.

4.10 Risk Analysis

The following section analyzes the risks involved in the development and deployment of the A.X.I.S. system across different domains.

4.10.1 Technical Risks

- **API and Protocol Stability:** A.X.I.S. relies heavily on third-party APIs (Slack, GitHub). Updates to these external platforms, or changes in how target IDEs (like Claude Code or Cursor) implement MCP, could disrupt the prompt interception pipeline.
- **Data Ingestion Rate Limits:** Continuous extraction of organizational data from Slack and GitHub may encounter API rate limits. This could temporarily delay the synchronization of the traceability graph, causing the AI to operate on slightly outdated architectural context.

4.10.2 Programmatic Risks

- **Semantic Validation Accuracy:** The system uses a Large Language Model (LLM) to evaluate intercepted developer prompts against the RAG database. There is a risk of the LLM misinterpreting the prompt's intent, leading to "false positives" (flagging a completely valid developer action) or "false negatives" (failing to catch an out-of-order execution).
- **Latency Overhead:** Intercepting a prompt, querying the vector database, evaluating the rules, and injecting the context must happen in near real-time. If the semantic validation process takes too long, it will introduce latency into the IDE, degrading the developer's coding experience.

4.10.3 Security Risks

- **Sensitive Data Handling:** Because A.X.I.S. continuously ingests internal organizational discussions and proprietary project requirements, there is a risk associated with data privacy. Ensuring the secure storage of this data within the database (Supabase) and secure transmission during MCP interception is critical to prevent exposing confidential company information.

4.11 Conclusion

This chapter specified the requirements of A.X.I.S. It listed the key features of the system and its functional requirements, covering account and access management, data ingestion, architecture management, prompt checking and auditing. The quality attributes and measurable non-functional requirements defined the expected performance, usability, reliability, security and portability of the system, along with

the assumptions behind them. The use cases and graphical user interfaces described how technical leads, project managers and developers interact with A.X.I.S., while the database design presented its entities through an ER diagram and data dictionary. Finally, the hardware and software requirements were specified and the technical, programmatic and security risks of the project were analyzed, providing a clear basis for the system design presented in the next chapter.

Chapter 5 High-Level and Low-Level Design

The complete details of high-level and low-level design of the system we are building are provided in this chapter.

5.1 System Overview

An AI-governed architectural middleware program called A.X.I.S. was created to close the gap between autonomous AI code development and organizational agreement. To help AI coding agents follow the agreed structural boundaries and development phases, the system functions as a guardrail, intercepting developer inputs.

5.1.1 Continuous Data Ingestion Engine

The system incorporates an automated ingestion pipeline that connects directly to organizational platforms like Slack and GitHub. It extracts unstructured data, such as team discussions, pull request reviews, and project requirements, transforming them into a structured format for architectural governance.

5.1.2 Traceability Graph and Vector Storage

To translate fragmented consensus into enforceable rules, A.X.I.S. utilizes a mathematically searchable traceability graph. Powered by Retrieval-Augmented Generation (RAG) and a vector database (Supabase with `pgvector`), this module indexes the ingested data so that architectural constraints can be semantically retrieved in real-time.

5.1.3 Model Context Protocol (MCP) Server

At the core of the system is the MCP Server, which connects the developer's coding agent (e.g., Cursor) to the team's project knowledge. It receives the developer's prompt, validates it against the traceability graph, and either supplies the relevant context to the coding agent or, if the prompt violates a constraint, asks the developer to confirm whether to proceed and records the decision.

5.1.4 Visual Architecture Studio

The platform provides a highly minimal, node-based interactive canvas (built with React Flow) wrapped in a Tauri desktop environment. This allows Technical Leads to manually design, edit, and visualize

structural dependencies and developmental timelines, which are instantly converted into system constraints.

5.2 Design Considerations

This section outlines important considerations that need to be addressed during the design phase of A.X.I.S. to ensure seamless integration between AI agents and developer workflows.

5.2.1 Assumptions and Dependencies

The design of A.X.I.S. relies on the following assumptions and dependencies:

- Developers use MCP-compatible agents such as Claude Code, Cursor or Antigravity.
- Semantic checks depend on a third-party LLM API (Gemini).
- The desktop app runs on Windows, macOS and Linux.
- Users understand basic software architecture concepts.
- The MCP specification and third-party API limits may change.

5.2.2 General Constraints

A.X.I.S. operates within the following constraints:

- The RAG and MCP back-ends run on a server, not on user machines.
- The MCP server must follow the JSON-RPC format defined by the protocol.
- Only hashed API keys are stored, and OAuth tokens are encrypted.
- Prompt checks should finish within 5 seconds.
- Slack, GitHub and LLM rate limits require caching and retries.

5.2.3 Goals and Guidelines

The design of A.X.I.S. follows these goals and guidelines:

- The interface is a minimal dark theme that feels familiar to developers.
- Fast, focused context is preferred over large data retrievals.
- Rules are checked strictly, and every violation asks the developer to confirm.

5.2.4 Development Methods

The development of A.X.I.S. follows these methods:

- The front-end is built from reusable React components.
- The team works in Agile iterations to adapt to API and MCP changes.
- Data structures such as the traceability graph are specified before coding.

5.3 System Architecture

The system architecture of A.X.I.S. is given in Figure 5.1, which highlights how the system collects project knowledge, processes it and interacts with users and external systems.

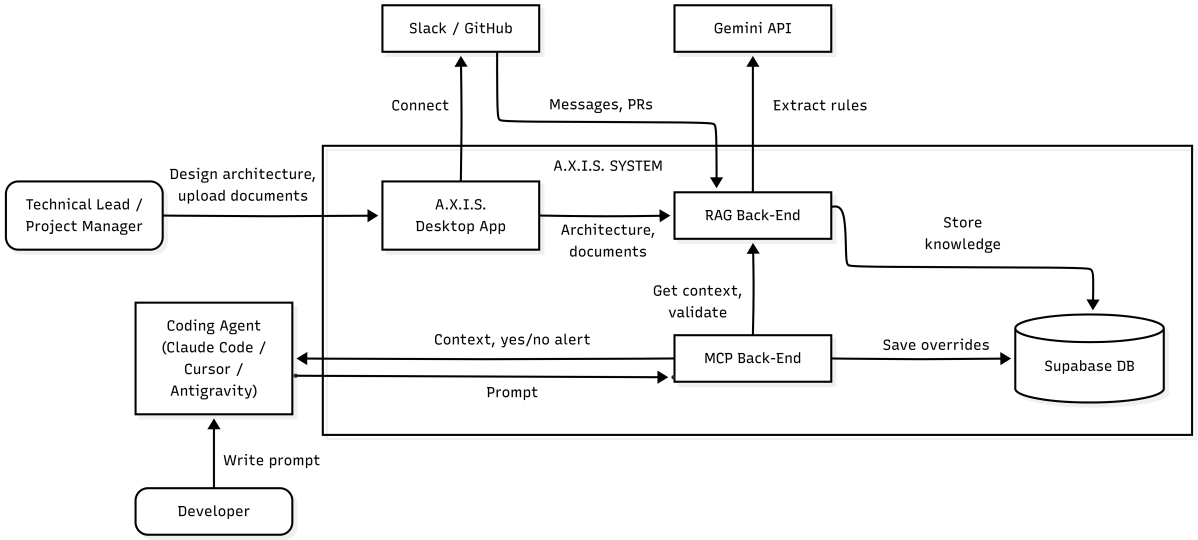


Figure 5.1: Architecture Diagram of A.X.I.S.

The architecture of A.X.I.S. is designed to bring scattered project knowledge into one place and deliver it to AI coding agents while code is being written. It consists of four subsystems within the system boundary: the A.X.I.S. Desktop App, the RAG Back-End, the MCP Back-End and the Supabase database. The users, Slack, GitHub, the coding agents and the Gemini API lie outside this boundary as external entities.

Top-Level Responsibilities

At the top-most level, A.X.I.S. is responsible for the following major functionalities:

- **Project Management:** Handles user authentication, projects, architecture design and developer API keys.
- **Knowledge Management:** Collects knowledge from Slack, GitHub and documents, and main-

tains it as a traceability graph.

- **Agent Integration:** Delivers relevant context to coding agents and alerts developers about constraint violations.
- **Data Storage:** Stores application data, the traceability graph and the developer decision log.

Rationale

The decomposition is driven by the need for performance and modularity. Knowledge processing is heavy and runs in the background, while responding to coding agents must be fast, so these are handled by separate back-ends that can work and scale independently. A single back-end was rejected because heavy processing would delay responses to developers. Retrieval logic is kept only in the RAG Back-End, so that there is a single source of truth for project knowledge. The system follows a layered architecture, with the MCP Back-End acting as a facade that provides a single interface to all coding agents.

5.3.1 Subsystem Architecture

5.3.1.1 A.X.I.S. Desktop App

This subsystem provides the interface through which technical leads and project managers set up and manage their projects.

Internal Components:

- Authentication Module
- Project and API Key Management
- Architecture Canvas
- Integrations and Document Upload

Interactions:

- User logs in and creates a project.
- User designs the architecture and uploads documents.
- User connects Slack and GitHub and generates API keys for developers.

5.3.1.2 RAG Back-End

This subsystem is responsible for collecting, processing, storing, and retrieving project knowledge.

Internal Components:

- Ingestion Service
- Extraction and Consolidation Service
- Retrieval Service
- Semantic Linter

Interactions:

- Messages, pull requests, documents, and architecture are received.
- Rules are extracted using the Gemini API and outdated ones are marked as superseded.
- Relevant context and constraint violations are returned to the MCP Back-End.

5.3.1.3 MCP Back-End

This subsystem acts as the single point of contact between A.X.I.S. and the developers' coding agents.

Internal Components:

- MCP Server
- API Key Verification
- Human-in-the-Loop Guard

Interactions:

- Coding agent sends the developer's prompt.
- MCP Back-End requests context and validation from the RAG Back-End.
- Context is returned to the coding agent, along with a yes/no alert if a violation is found.
- Developer's override decision is saved in the database.

5.3.1.4 Supabase Database

This subsystem stores all system data in a single PostgreSQL database with the `pgvector` extension.

Internal Components:

- Application Data
- Traceability Graph and Embeddings
- Decision Log

Interactions:

- RAG Back-End stores and searches project knowledge.
- MCP Back-End records developer override decisions.

5.4 Architectural Strategies

This section describes the strategies that affect the overall organization of A.X.I.S. and its higher-level structures, along with the alternatives that were considered and the reasons for each choice.

5.4.1 Choice of Technologies

- **React and React Flow for the front-end:** React was chosen for its component-based design and large ecosystem, and React Flow for building the interactive architecture canvas. The web application is wrapped into a standalone desktop application, so a single codebase runs on Windows, macOS and Linux instead of separate native applications for each operating system.
- **Python for the back-end:** Python was chosen for both back-end systems because of its strong support for AI, embeddings and the FastMCP framework used to build the MCP server.
- **Supabase (PostgreSQL with pgvector) for the database:** Chosen because it stores relational data, the traceability graph and vector embeddings in one database, and provides built-in authentication.

5.4.2 Reuse of Existing Software Components

- **FastMCP:** Used to implement the Model Context Protocol instead of writing the protocol from scratch.
- **Gemini AI services:** Used for decision extraction and constraint checking through a common interface, so that the provider can be changed without modifying the rest of the system. Hosting a large language model locally was rejected due to its hardware requirements.
- **fastembed:** Used to generate embeddings locally on the CPU, which avoids API costs and allows the system to run on standard laptops without a GPU.
- **Supabase Auth:** Used for user authentication instead of building a custom login system.

5.4.3 External Database and Data Storage Management

All data is stored in a single Supabase database, with the traceability graph represented as knowledge nodes and trace links. Using a separate graph database and a separate vector database was rejected,

because keeping three data stores synchronized would add complexity without a clear benefit at the expected scale.

5.4.4 Communication Mechanisms

The front-end communicates with the back-end systems through HTTPS REST APIs. AI coding agents connect to A.X.I.S. through the Model Context Protocol, an open standard supported by Claude Code, Cursor and Antigravity. Building a separate plugin for each agent was rejected, as it would require maintaining several codebases and tie the system to specific vendors. Slack and GitHub send new events to A.X.I.S. through the Slack Events API and GitHub webhooks, instead of A.X.I.S. repeatedly polling them.

5.4.5 Concurrency and Synchronization

Incoming Slack and GitHub events are processed asynchronously in the background, so that ingestion does not slow down requests from coding agents or the front-end. The MCP server handles requests from multiple developers at the same time, and a periodic synchronization job recovers any events that were missed.

5.4.6 User Interface Paradigms

When a request conflicts with a constraint, the developer is asked a yes or no question directly inside the coding agent, using MCP elicitation where supported and the agent's chat otherwise. An approval dashboard for administrators was rejected, as it would interrupt the developer's workflow. The front-end follows Shneiderman's Eight Golden Rules of Interface Design to keep the canvas and settings consistent and easy to use.

5.4.7 Error Detection and Recovery

If A.X.I.S. is unavailable, the coding agents continue to work normally and the developer is informed that the organizational context could not be retrieved. Blocking the agent until A.X.I.S. recovers was rejected, since a failure in the system should never stop development. Failed ingestion tasks are retried automatically and all errors are logged for analysis.

5.4.8 Future Plans for Extension

The system is divided into a front-end, a RAG back-end and an MCP back-end, so that each can be developed, tested and extended independently. Each knowledge source is implemented as a separate

connector, allowing new sources such as Jira or Confluence to be added later without changing the core system.

5.4.9 Version Control

The source code is hosted in repositories on GitHub. Team members work on separate branches and merge their changes through pull requests, which are reviewed by another team member before approval.

5.5 Class Diagram

Figure 5.2 shows the main classes of A.X.I.S. and the relationships between them.

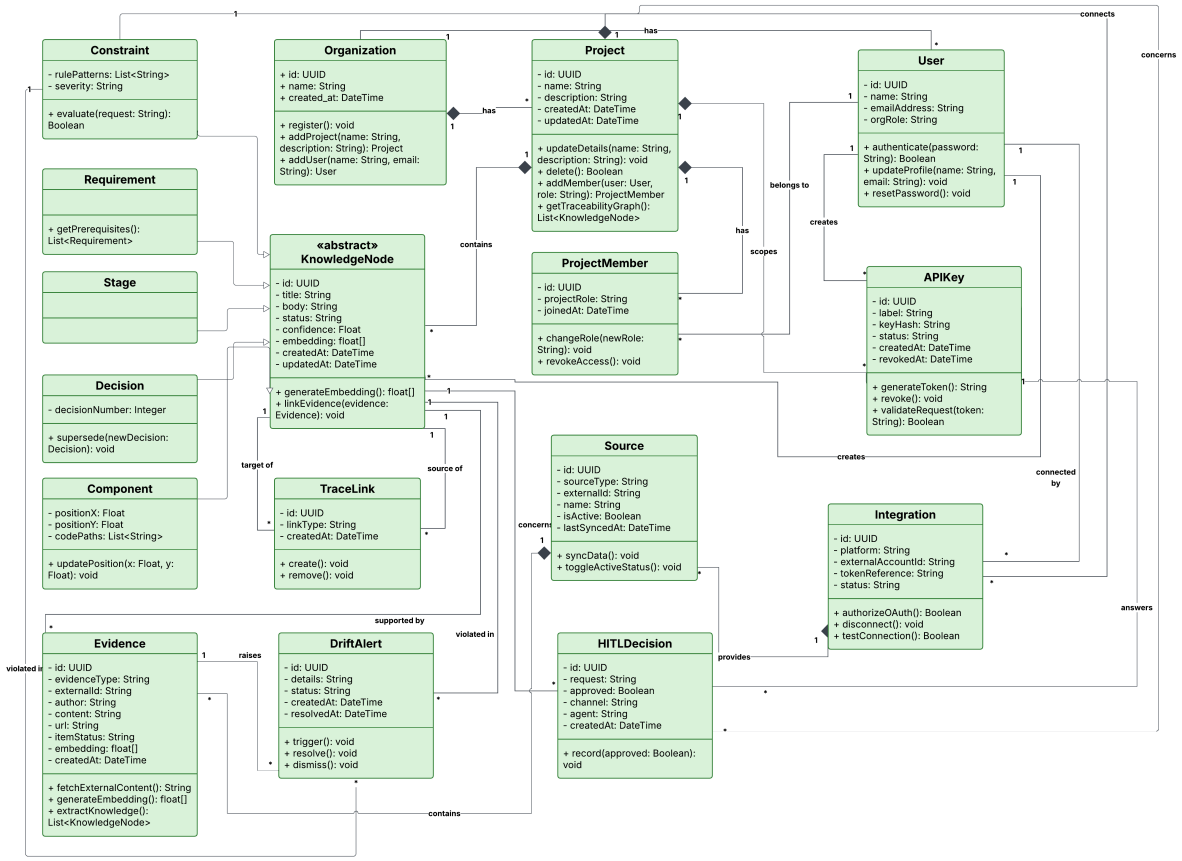


Figure 5.2: Class Diagram

5.6 Sequence Diagrams

The following section displays the sequence diagrams modeling the core behavioral workflows and system interactions of the A.X.I.S. platform.

5.6.1 Standard User Authentication

Figure 5.3 shows how a technical lead or project manager logs in. The front-end sends the credentials to the authentication controller, which looks up the user, verifies the password hash and, if it matches, returns a session token and redirects the user to the dashboard.

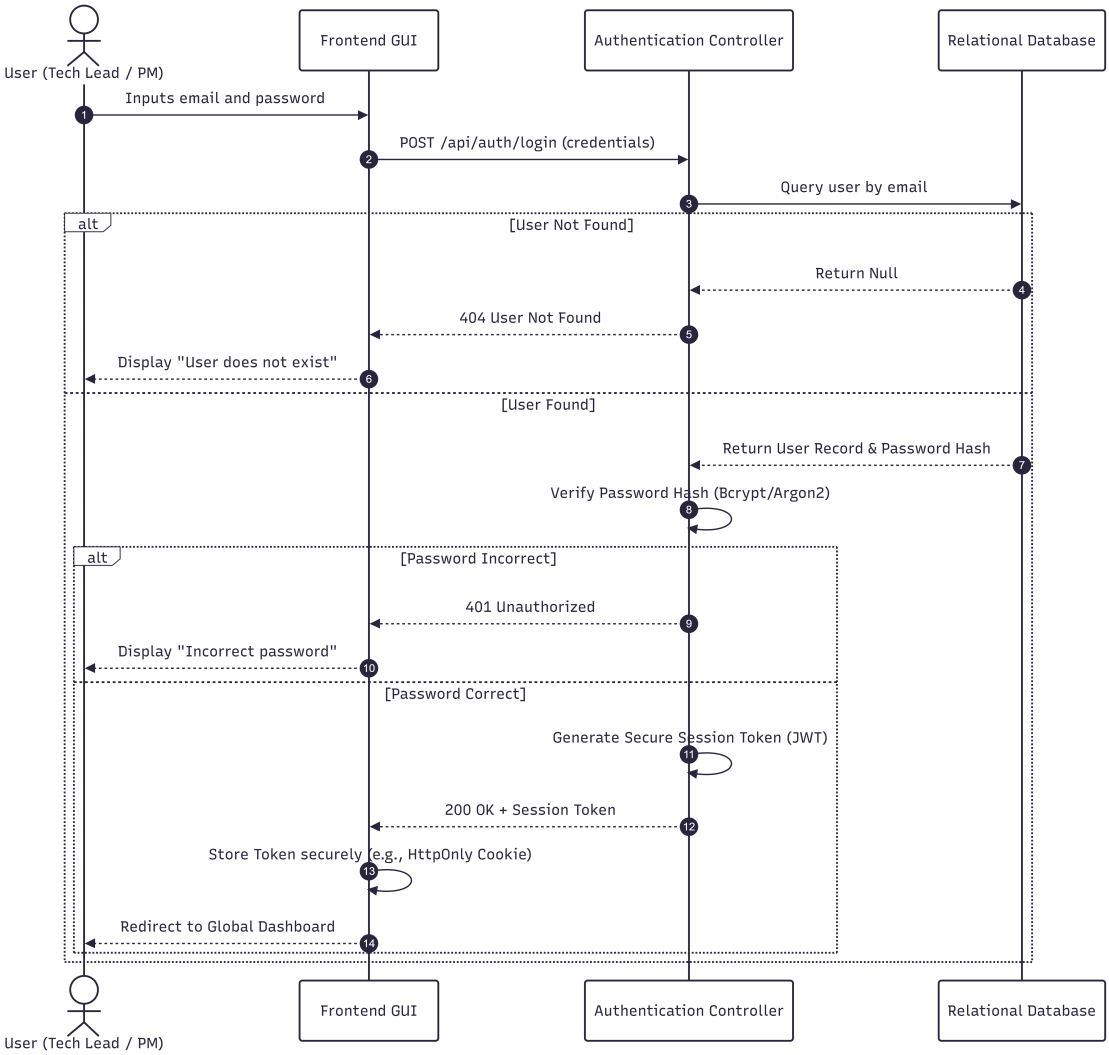


Figure 5.3: Standard User Authentication

5.6.2 Document Parsing and Knowledge Extraction

Figure 5.4 shows how an uploaded SRS document is processed. After its format is validated, the text is split into chunks, the LLM identifies requirements and constraints, and these are stored as knowledge nodes before the user is notified.

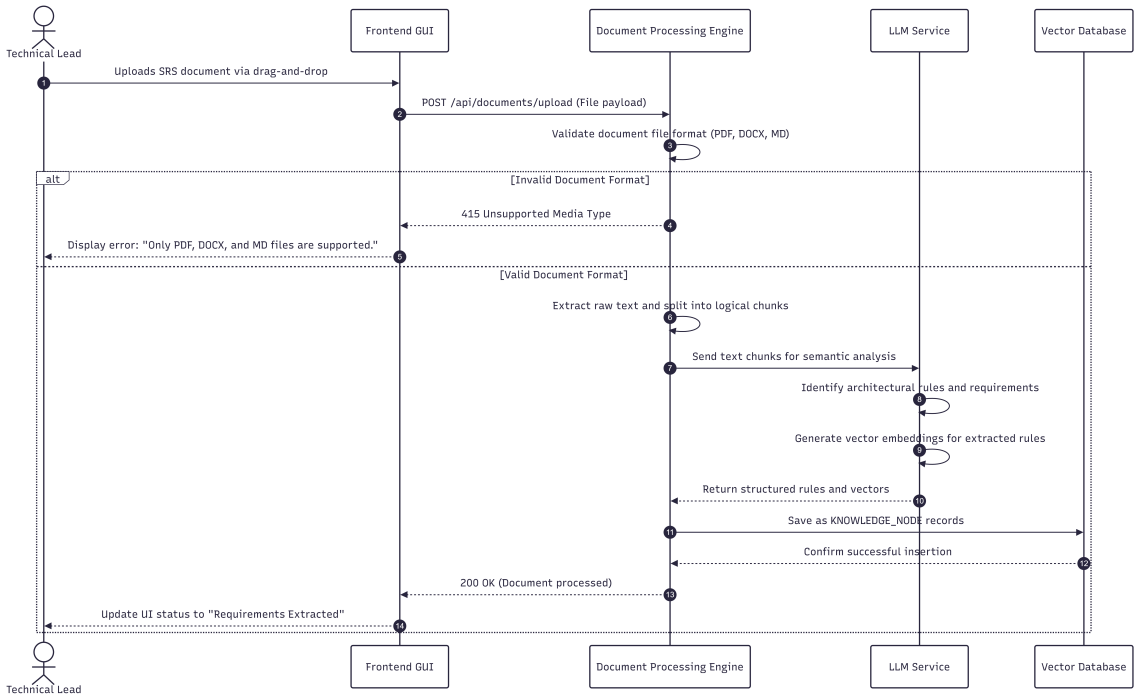


Figure 5.4: Document Parsing and Knowledge Extraction

5.6.3 Continuous Data Ingestion

Figure 5.5 shows how new Slack and GitHub activity reaches A.X.I.S. through webhooks. The webhook listener verifies the request, acknowledges it immediately and forwards the event for asynchronous processing, after which the extracted context is stored as evidence.

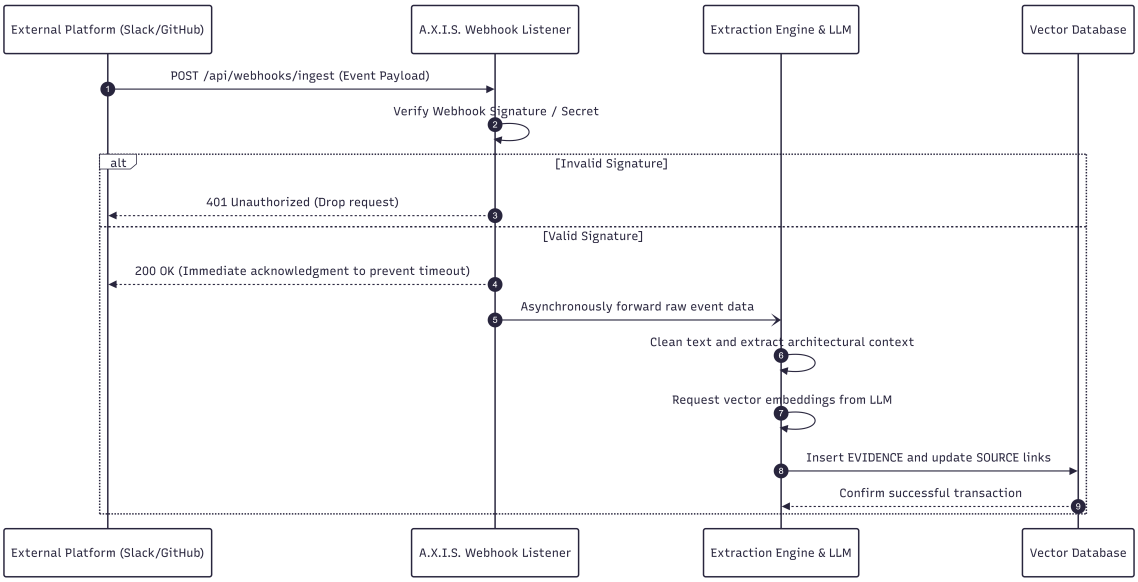


Figure 5.5: Continuous Data Ingestion

5.6.4 Visual Architecture to Constraint Generation

Figure 5.6 shows how a saved canvas becomes constraints. Structural designs are checked for unique component names, developmental flows are checked for circular dependencies, and the resulting constraints are committed to the traceability graph as a new version.

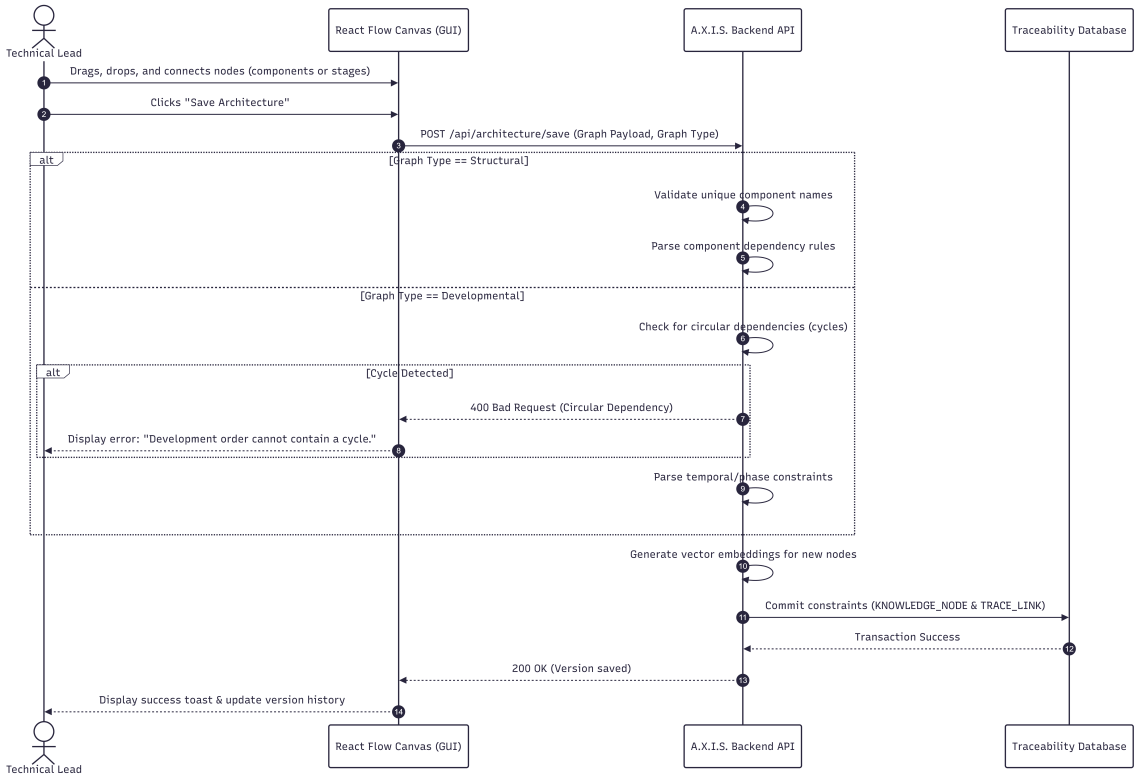


Figure 5.6: Visual Architecture to Constraint Generation

5.6.5 MCP Client Authentication and Handshake

Figure 5.7 shows how a developer's coding agent connects to A.X.I.S. The gateway hashes the provided API key and checks it against the stored record, rejecting invalid or revoked keys and opening an MCP session for valid ones.

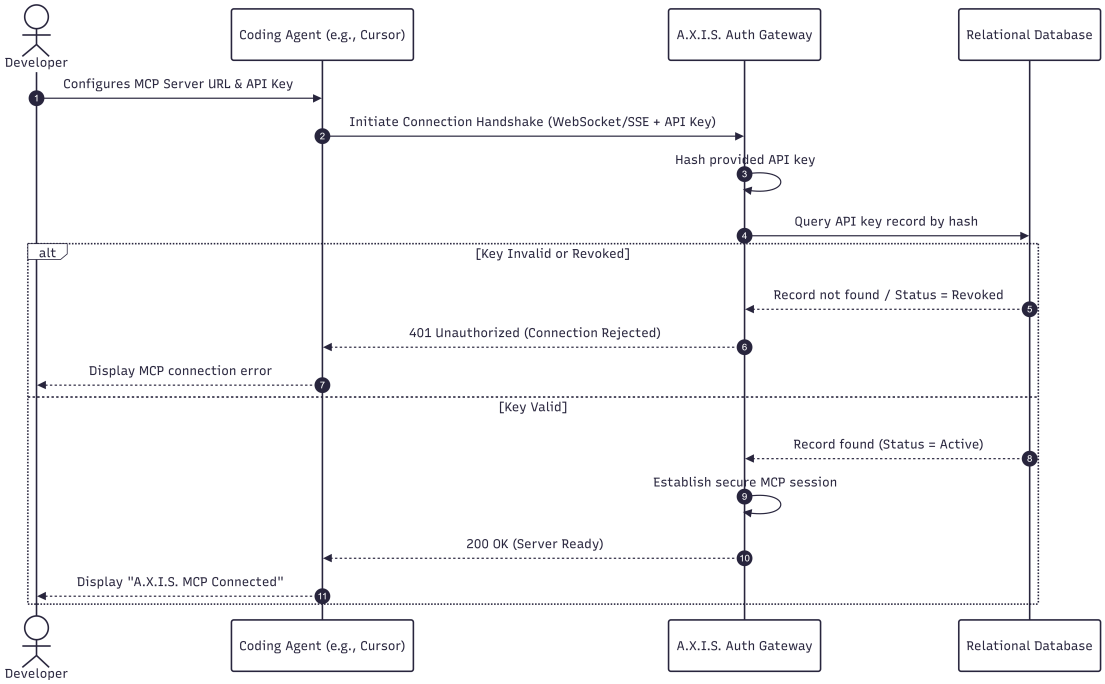


Figure 5.7: MCP Client Authentication and Handshake

5.6.6 AI Prompt Interception and Semantic Validation

Figure 5.8 shows how a developer's prompt is checked. The MCP server retrieves the active constraints, evaluates the prompt against them, and either supplies the relevant context to the coding agent or alerts the developer when a constraint is violated.

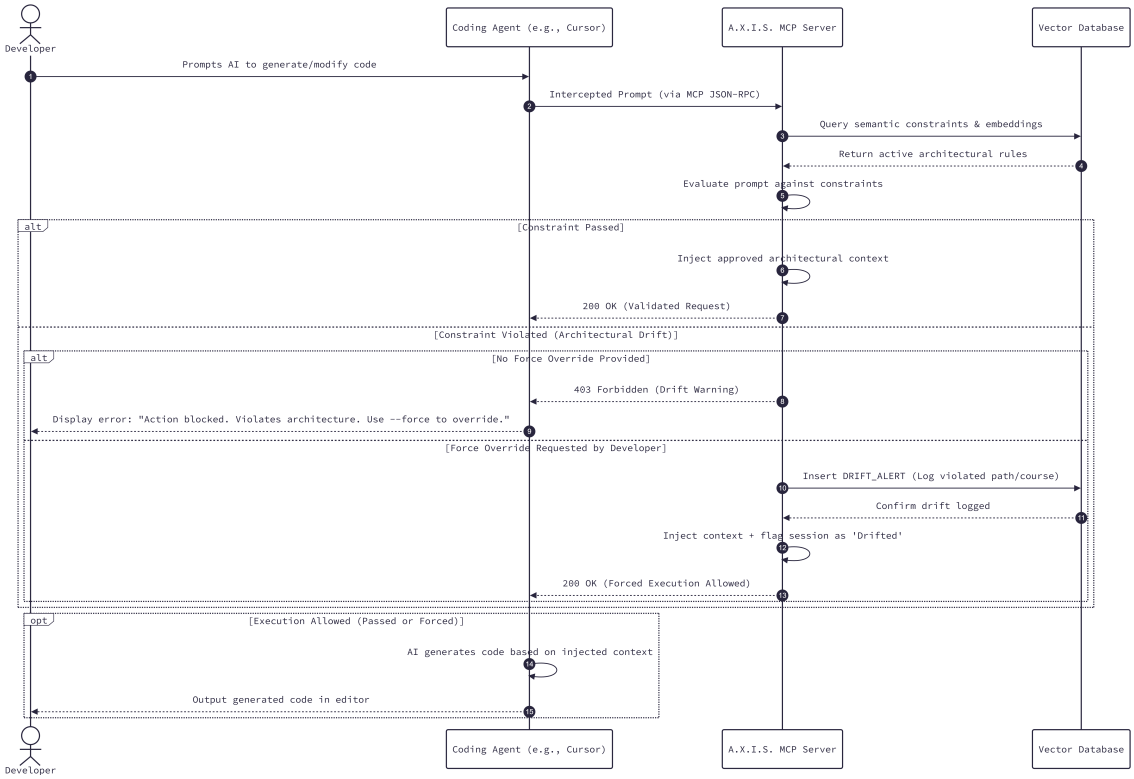


Figure 5.8: AI Prompt Interception and Semantic Validation

5.7 Policies and Tactics

The following policies and tactics will be followed during the development of A.X.I.S. to ensure that the system is consistent, reliable and easy to maintain.

5.7.1 Hybrid Retrieval with Graph Expansion

Relevant context is found by combining keyword search and vector similarity search, and then following trace links to related decisions, requirements and components. Pure vector search was rejected because it misses exact terms such as service and database names, and cannot find constraints that are only connected to a task through the graph. Fine-tuning an LLM on team data was rejected because it is costly and cannot keep up with frequently changing decisions.

5.7.2 Coding Guidelines and Conventions

The back-end code follows the Python PEP 8 standard, with type hints and docstrings for all public functions. The front-end code follows standard TypeScript conventions, enforced automatically using ESLint and Prettier. Visual Studio Code is used as the main development environment for all team members.

5.7.3 MCP Tool Interface Design

The MCP server exposes a small, fixed set of tools, each with a clear name, description and input parameters, so that coding agents can understand when to use them. Every tool returns structured data, and every constraint or decision it returns includes a link to its source, so that the developer can verify it. The server also provides instructions that guide agents to check requests and retrieve context before writing code.

5.7.4 Prompt and Model Management

All prompts sent to the AI service are stored as version-controlled files rather than written directly in the code, so that changes to them can be reviewed and tracked. The AI service is asked to return structured output in a fixed format, which is validated before being stored. The AI provider and model are set through configuration, allowing them to be changed without modifying the source code.

5.7.5 Testing Strategy

Unit testing and integration testing will be used for individual modules and the communication between subsystems, along with black box testing of the complete system. Since the output of the AI components

cannot be checked through normal functional tests alone, an automated set of test prompts with known expected results will be used to verify context retrieval, constraint checking and human-in-the-loop confirmation. The system will also be tested manually with Claude Code, Cursor and Antigravity.

5.7.6 Security and Secrets Management

API keys and database credentials are stored in environment files on the server and are never committed to version control or sent to the client. Access to Slack and GitHub is requested through OAuth with only the minimum read permissions required, and Supabase row-level security ensures that each organization can only access its own data.

5.7.7 Source Code Organization

The source code is organized into separate directories for the front-end, the RAG back-end, the MCP server and the database migrations, with documentation kept in a separate folder. Each directory is further divided by functionality, so that team members can work on different subsystems independently.

5.8 Conclusion

This chapter presented the high-level and low-level design of A.X.I.S. The system overview and design considerations described its main modules, along with the assumptions, constraints and goals that shaped its design. The system architecture divided A.X.I.S. into four subsystems, the Desktop App, the RAG Back-End, the MCP Back-End and the Supabase database, separating heavy knowledge processing from fast responses to coding agents. The architectural strategies justified key decisions, such as using a single database for relational data, the traceability graph and embeddings, connecting to coding agents through the open Model Context Protocol instead of separate plugins, and allowing developers to keep working if A.X.I.S. is unavailable. The class diagram modeled the structure of the system, while the sequence diagrams showed its core workflows, from authentication and data ingestion to prompt checking. Finally, the policies and tactics defined how context will be retrieved and how the code, prompts and secrets will be managed and tested, providing a clear plan for implementation.

Bibliography

[1] Eraser, "Diagrams as Code." Available: https://www.eraser.io/. [Accessed September 04, 2026].

[2] Miro, "The Visual Workspace for Innovation." Available: https://miro.com/. [Accessed September 04, 2026].

[3] Miro, "Miro Launches MCP Server to Connect Visual Collaboration with AI Coding Tools." Available: https://miro.com/newsroom/miro-launches-mcp-server-to-connect-visual-collaboration-with-ai-coding/. [Accessed October 03, 2026].

[4] Structurizr, "Structurizr - Architecture as Code." Available: https://structurizr.com/. [Accessed September 04, 2026].

[5] Tessl, "Tessl - Agent Enablement Platform." Available: https://tessl.io/. [Accessed October 03, 2026].

[6] Kiro, "Kiro - Agentic Development Environment." Available: https://kiro.dev/. [Accessed September 04, 2026].

[7] Significant Gravitas, "AutoGPT." Available: https://github.com/Significant-Gravitas/AutoGPT. [Accessed October 02, 2026].

[8] Y. Nakajima, "BabyAGI." Available: https://github.com/yoheinakajima/babyagi. [Accessed October 02, 2026].

[9] Jama Software, "Jama Connect: Requirements Management and Live Traceability." Available: https://www.jamasoftware.com/. [Accessed October 02, 2026].

[10] Jama Software, "MCP vs. REST API: What's the Difference." Available: https://support.jamasoftware.com/hc/en-us/articles/47235506164749-MCP-vs-REST-API-What-s-the-Difference. [Accessed October 09, 2026].

[11] Atlassian, "Rovo: AI-Powered Search, Chat and Agents." Available: https://www.atlassian.com/software/rovo. [Accessed October 02, 2026].

[12] Atlassian, "Atlassian Opens Up Its Teamwork Graph to Power Agentic Work Across the Enterprise." Available: https://www.nasdaq.com/press-release/atlassian-opens-its-teamwork-graph-power-agentic-work-across-enterprise-2026-05-06. [Accessed October 02, 2026].

[13] Unblocked, "Unblocked: The Context Layer for Agentic Software Development." Available: https://getunblocked.com/ai-info/. [Accessed October 10, 2026].

[14] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory." arXiv:2501.13956, 2025. Available: https://arxiv.org/abs/2501.13956.

[15] Augment Code, "Context Engine MCP Now Generally Available." Available: https://www.augmentcode.com/. [Accessed October 10, 2026].

[16] CodeRabbit, "Knowledge Base." Available: https://docs.coderabbit.ai/knowledge-base. [Accessed October 10, 2026].

[17] Glean, "Introducing MCP in Glean: From Connectivity to Enterprise-Ready Actions." Available: https://glean.com/blog/mcp-mar-drop-2026. [Accessed October 10, 2026].

[18] Glean, "About Glean MCP Server." Available: https://docs.glean.com/administration/platform/mcp/about. [Accessed October 10, 2026].

[19] TNG Technology Consulting, "ArchUnit User Guide." Available: https://www.archunit.org/userguide/html/000_Index.html. [Accessed October 10, 2026].